



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Swift Contest: Progettazione e Implementazione di un'Applicazione Cross-Platform per la Gestione di Contest

RELATORE

Prof. Rita Francese

Università degli Studi di Salerno

CANDIDATO

Tommaso Moretta

Matricola: 0512118903

Anno Accademico 2024-2025

*“Don’t believe what you believe just ‘cause
that’s how they raised you”*

— NF

Estratto

La gestione tradizionale di contest, basata su strumenti eterogenei, è un processo frammentato e insicuro che compromette l'integrità dei dati e il coordinamento in tempo reale. L'analisi dello stato dell'arte ha confermato un **vuoto di mercato** per soluzioni verticali, motivando la progettazione di **Swift Contest**: una piattaforma software *end-to-end*, sicura e multiplatforma.

Per soddisfare i requisiti emersi dall'analisi, è stato selezionato uno stack tecnologico moderno. L'esigenza di sviluppo *cross-platform* ha guidato la scelta del framework **Flutter**, mentre la necessità di un backend robusto e relazionale ha portato all'adozione di **Supabase (PostgreSQL)**. La logica serverless è stata implementata in modo strategico: le **Funzioni RPC** in PL/pgSQL garantiscono l'**atomicità delle transazioni**, mentre le **Edge Functions** in TypeScript gestiscono le **interazioni con API esterne e con lo Storage**.

La progettazione del sistema si fonda su un'architettura sicura basata sul **Principio del Minimo Privilegio (PoLP)**, dove la **Row Level Security (RLS)** e un'API **controllata** lavorano in sinergia per mediare ogni accesso ai dati. Sul frontend, il pattern **MVVM con BLoC** garantisce la separazione delle responsabilità, mentre sul backend, il **meccanismo di snapshot** assicura l'integrità del voto. Un sistema di **autenticazione ibrido** supporta inoltre l'accesso sicuro per "Giurati Ospiti" tramite autenticazione anonima.

La fase finale del progetto ha visto la distribuzione delle sue componenti: le **migrazioni IaC e le funzioni serverless** sono state rilasciate sull'infrastruttura **Supabase**; il client web è stato distribuito su **Vercel** tramite pipeline di CI/CD; e l'artefatto Android su **GitHub**, culminando in un **prototipo completo e funzionante**. In conclusione, Swift Contest non solo valida la fattibilità dell'approccio architetturale, ma si configura come un **blueprint robusto e scalabile** per la creazione di applicazioni collaborative sicure, dimostrando come un ecosistema tecnologico moderno possa colmare una reale esigenza di mercato.

Abstract

The management of contests, based on disjointed tools, is a fragmented and insecure process that compromises data integrity and real-time coordination. The state-of-the-art analysis confirmed a **market gap** for vertical solutions, motivating the design of **Swift Contest**: a secure, cross-platform, and *end-to-end* software platform.

To meet the requirements identified during the analysis, a modern technology stack was selected. The need for *cross-platform* development led to the choice of the **Flutter** framework, while the requirement for a robust, relational backend prompted the adoption of **Supabase (PostgreSQL)**. Serverless logic was implemented strategically: **RPC Functions** in PL/pgSQL ensure the **atomicity of database transactions**, while **Edge Functions** in TypeScript handle **interactions with external APIs and Storage**.

The system's design is founded on a secure architecture based on the **Principle of Least Privilege (PoLP)**, where **Row Level Security (RLS)** and a **controlled API** work in synergy to mediate all data access. On the frontend, the **MVVM with BLoC** pattern ensures a separation of concerns, while on the backend, a **snapshot mechanism** guarantees voting integrity. A **hybrid authentication system** also supports secure access for "Guest Jurors" via anonymous authentication.

The final project phase involved deploying its components: **IaC migrations and serverless functions** were released to the **Supabase** infrastructure; the web client was deployed on **Vercel** via CI/CD pipelines; and the Android artifact on **GitHub**, culminating in a **complete and working prototype**. In conclusion, Swift Contest not only validates the feasibility of the architectural approach but also stands as a **robust and scalable blueprint** for building secure collaborative applications, demonstrating how a modern technology ecosystem can fill a tangible market need.

Indice

Elenco delle figure	v
Elenco delle tabelle	vii
Elenco dei listati	viii
1 Introduzione	1
1.1 Motivazioni e contesto	1
1.2 Obiettivi del progetto	2
1.3 Contributi progettuali e ingegneristici	3
1.4 Struttura del documento	4
2 Tecnologie e strumenti utilizzati	5
2.1 Framework, piattaforme e linguaggi	5
2.1.1 Framework Frontend	6
2.1.2 Framework e piattaforme Backend	6
2.1.3 Linguaggi di programmazione	7
2.2 Strumenti, servizi e piattaforme di supporto	7
2.2.1 Piattaforme di distribuzione e servizi Cloud	8
2.2.2 Strumenti di sviluppo e amministrazione	8
2.2.3 Version Control e Code Hosting	9

2.2.4	Strumenti di documentazione e modellazione	9
3	Contesto e analisi del sistema	10
3.1	Il problema: frammentazione e inefficienza nella gestione dei contest	10
3.2	Stato dell'arte e analisi delle alternative	12
3.2.1	Strumenti generici adattati: lo "Status Quo"	12
3.2.2	Piattaforme per domini correlati	12
3.2.3	Posizionamento di Swift Contest	13
3.3	Attori e casi d'uso	13
3.3.1	Attori del sistema	14
3.3.2	Casi d'Uso: gestione account e funzionalità generali	15
3.3.3	Casi d'Uso: gestione del ciclo di vita del contest	16
3.3.4	Casi d'Uso: gestione delle votazioni e dei risultati	17
3.3.5	Casi d'Uso: amministrazione del sistema	18
3.4	Requisiti del sistema	18
3.4.1	Requisiti funzionali (Functionality)	18
3.4.2	Requisiti non funzionali	20
3.5	Modello del dominio	22
3.5.1	Entità fondamentali	23
3.5.2	Vincoli chiave del dominio	23
4	Progettazione e implementazione del sistema	25
4.1	Linee guida progettuali	25
4.2	Architettura del frontend: il pattern MVVM con BLoC	26
4.2.1	Implementazione del ViewModel: il pattern BLoC	28
4.2.2	Navigazione con AutoRoute	31
4.2.3	Strategia di navigazione e passaggio dei dati	32
4.2.4	Design system e theming	33
4.3	Progettazione del database	37
4.3.1	Schema fisico e tabelle principali	37
4.3.2	Scelte di design chiave	38
4.3.3	Implicazioni del design del form di voto sull'aggregazione dei risultati	43

4.4	Architettura del backend	45
4.4.1	Funzioni RPC (Remote Procedure Call)	45
4.4.2	Edge Functions	47
4.5	Architettura di sicurezza: Principio del Minimo Privilegio	50
4.5.1	Strategia di implementazione: centralizzazione della logica di autorizzazione	50
4.5.2	Il ruolo della RLS e dell'API come "Gatekeeper"	51
4.5.3	L'eccezione necessaria: le policy RLS per il servizio Realtime .	51
4.5.4	Misure di sicurezza aggiuntive	52
4.6	Progettazione e implementazione degli strumenti di amministrazione	55
4.6.1	Esigenza di un pannello di amministrazione	55
4.6.2	Scelta di Retool per lo sviluppo rapido	55
4.6.3	Architettura di comunicazione sicura tra Retool e Supabase .	56
4.6.4	Funzionalità implementate nel pannello	59
5	Distribuzione e prodotto finale	61
5.1	Strategia di distribuzione	61
5.2	Distribuzione web su Vercel	62
5.3	Distribuzione Android	62
5.4	Backend e Supabase	64
5.5	Dashboard amministrativa (Retool)	66
5.6	Prodotto finale: interfaccia utente e flussi operativi	66
5.6.1	Accesso e autenticazione	67
5.6.2	Gestione delle impostazioni	67
5.6.3	Flusso dell'organizzatore	69
5.6.4	Flusso del partecipante	76
5.6.5	Flusso del giurato	77
5.6.6	Flusso del giurato semplice (ospite)	79
6	Conclusioni	80
6.1	Sintesi del lavoro svolto	80
6.2	Risultati ottenuti	80
6.3	Conoscenze acquisite	81

6.4	Valutazione personale	82
	Riferimenti	84

Elenco delle figure

3.1	Confronto tra il flusso di lavoro tradizionale e l'approccio integrato di Swift Contest.	11
3.2	Diagramma dei casi d'uso: gestione account e funzionalità generali. .	15
3.3	Diagramma dei casi d'uso: gestione del ciclo di vita del contest. . . .	16
3.4	Diagramma dei casi d'uso: gestione delle votazioni e dei risultati. . .	17
3.5	Diagramma dei casi d'uso: amministrazione del sistema.	18
3.6	Diagramma delle classi concettuale.	22
4.1	Architettura Client-Server del sistema Swift Contest con backend BaaS.	26
4.2	Architettura del Client MVVM con BLoC.	27
4.3	Diagramma Entità-Relazioni (ERD) dello schema "public" del database.	37
4.4	Flusso di una chiamata sicura da Retool a una Funzione RPC.	56
4.5	Flusso di una chiamata sicura da Retool a una Edge Function, protetta da una chiave API.	58
5.1	Diagramma di distribuzione dell'architettura completa di Swift Contest.	62
5.2	Pagina di accesso dell'applicazione web distribuita su Vercel.	63
5.3	La dashboard di amministrazione sviluppata in Retool, che mostra l'elenco degli utenti registrati.	66
5.4	Le schermate del flusso di autenticazione dell'applicazione.	67

5.5	Le schermate per la gestione delle impostazioni dell'app e dell'account utente.	68
5.6	La home page dell'organizzatore.	69
5.7	Le tre schermate del form guidato per la creazione di un nuovo contest.	70
5.8	Le principali tab di gestione all'interno di un contest.	71
5.9	Le viste di dettaglio per i due tipi di giuria e la configurazione del form di voto.	72
5.10	I passaggi per la configurazione di una nuova sessione di voto.	73
5.11	La schermata di una sessione di voto attiva (live).	73
5.12	Visualizzazione dei voti inviati da un giurato, suddivisi per scope.	74
5.13	La pagina per la generazione della classifica di una giuria.	75
5.14	Le tre schermate del form guidato per la sottomissione di un lavoro.	76
5.15	La tab "Work" del partecipante dopo aver sottomesso il proprio lavoro.	76
5.16	Le schermate principali per il flusso del giurato nominato.	77
5.17	Le quattro fasi principali della procedura di votazione guidata per il giurato.	78
5.18	I tre passaggi per l'accesso di un giurato semplice (ospite) alla piattaforma.	79

Elenco delle tabelle

4.1	Elenco esemplificativo delle funzioni RPC del sistema.	46
4.2	Elenco esemplificativo delle Edge Functions del sistema.	48
5.1	Riepilogo delle componenti Supabase e del loro utilizzo nel progetto.	65

Elenco dei listati

4.1	Funzione helper per la gestione sicura delle chiamate al backend. . .	30
4.2	Estratto del sistema di routing protetto con “AuthGuard”.	31
4.3	Esempio di validatore centralizzato e suo utilizzo in un widget. . . .	33
4.4	Definizione dell’estensione “ColorSchemeX” per aggiungere colori personalizzati al tema.	35
4.5	Implementazione del loader non-intrusivo tramite Overlay e Dart Extensions.	36
4.6	Definizione della tabella “voting_session_participants” per lo snapshot dei dati dei partecipanti.	42
4.7	Definizione della tabella “voting_session_juries” per lo snapshot dei dati delle giurie.	42
4.8	Definizione della tabella “voting_session_jurors” per lo snapshot dei dati dei giurati.	43
4.9	Implementazione della funzione RPC “organizer_cancel_voting_session”, che mostra il pattern di controllo dei permessi lato server.	47
4.10	Estratto dalla Edge Function “organizer-invite-juror” che mostra l’invocazione dell’API di Resend.	50
4.11	Policy RLS di tipo SELECT per la tabella “voting_sessions”, necessaria per il servizio Realtime.	52

4.12 Validazione del chiamante all'interno di una funzione RPC per l'amministrazione.	57
4.13 Validazione della chiave API all'interno di una Edge Function amministrativa.	59

CAPITOLO 1

Introduzione

1.1 Motivazioni e contesto

La gestione di *contest* in tempo reale rappresenta a oggi un processo frammentato e complesso. Organizzatori, partecipanti e giurati spesso ricorrono a strumenti diversi: servizi di posta elettronica per la comunicazione degli inviti, piattaforme di file sharing per la raccolta dei lavori, fogli di calcolo per la tabulazione dei punteggi.

Questa frammentazione genera inefficienze operative, rischi di errore umano e una significativa difficoltà nel garantire trasparenza e sicurezza. L'analisi dello stato dell'arte, come verrà dettagliato nel **Capitolo 3**, ha inoltre rivelato un **significativo vuoto di mercato**: non esistono soluzioni verticali e accessibili che coprano in modo integrato l'intero ciclo di vita di un *contest*.

Il progetto **Swift Contest** nasce proprio per colmare questa lacuna, proponendo un sistema moderno, sicuro e multiplatforma che unisce in un'unica applicazione le principali fasi del ciclo di vita di un *contest*: dalla creazione e configurazione, all'invito dei partecipanti e dei giurati, fino alla raccolta dei lavori e alla gestione delle sessioni di voto in tempo reale.

1.2 Obiettivi del progetto

L'obiettivo primario di questo lavoro è stato quello di **progettare e implementare l'architettura** di un sistema completo di *contest management*, adottando tecnologie moderne e approcci architetturali rigorosi.

Gli obiettivi specifici, che hanno guidato la progettazione, possono essere così sintetizzati:

- **Sviluppo cross-platform:** garantire un'esperienza utente fluida e unificata su più piattaforme (mobile e web) attraverso lo sviluppo di un client basato su un'unica codebase, un obiettivo reso possibile dall'adozione del framework Flutter [1].
- **Sicurezza "by design":** adottare un'architettura di sicurezza basata sul **Principio del Minimo Privilegio (Principle of Least Privilege)**, dove l'accesso al database è negato per default e concesso selettivamente solo tramite un'API controllata.
- **Esperienza utente mirata:** progettare interfacce utente intuitive e funzionali, modellate sui ruoli specifici dei diversi attori del sistema (organizzatore, partecipante, giurato).
- **Accesso flessibile alla votazione:** implementare un sistema di voto che supporti due modalità di accesso per i giurati. Oltre ai **giurati nominati** (utenti registrati e invitati), il sistema deve permettere la partecipazione di **giurati semplici** tramite un token di accesso sicuro, abbassando così la barriera d'ingresso per i giurati occasionali.
- **Affidabilità e auditabilità del processo:** assicurare l'integrità e la verificabilità del processo di voto attraverso meccanismi tecnici robusti, come lo *snapshot* dei dati al momento dell'avvio della sessione e la possibilità di applicare restrizioni basate sulla geolocalizzazione.
- **Gestione e moderazione del sistema:** realizzare una dashboard amministrativa per la gestione degli utenti e la moderazione dei contenuti.

1.3 Contributi progettuali e ingegneristici

L'obiettivo primario di questo lavoro di tesi non è di natura teorica, ma **pratica e orientata al prodotto**: la creazione di una soluzione completa e funzionante per un problema reale e non indirizzato dal mercato attuale. Il valore del progetto risiede quindi nella dimostrazione di come, partendo da un'esigenza concreta, sia stato possibile progettare e realizzare un sistema software complesso, utilizzando principi di ingegneria moderni come mezzo per garantire la qualità e la robustezza del risultato finale.

Il progetto **Swift Contest** si presenta quindi come un *blueprint*, ovvero un modello di riferimento, per la creazione di applicazioni collaborative complesse e sicure. I contributi specifici possono essere così riassunti:

- **Proposta e validazione di una nuova categoria di prodotto**: data l'assenza di soluzioni verticali, il progetto stesso si configura come un caso di studio e una validazione pratica per una nuova categoria di software dedicato alla gestione dei *contest*.
- **Dimostrazione di un'architettura sicura su BaaS**: il sistema dimostra come implementare un'architettura basata sul Principio del Minimo Privilegio su una piattaforma **Backend-as-a-Service (BaaS)** come Supabase [2], ma **centralizzando la logica di autorizzazione in API serverless (RPC ed Edge Functions)** invece di affidarsi unicamente a regole a livello di database (RLS).
- **Modello dati robusto per processi immutabili**: viene presentato un design del database che garantisce l'integrità e l'attendibilità dei processi di voto tramite l'uso di *attributi di snapshot* e denormalizzazione controllata.

In sintesi, il valore della tesi è nel dimostrare l'intero processo ingegneristico, dall'analisi di un vuoto di mercato alla progettazione e realizzazione di una soluzione *completa, sicura e ben ingegnerizzata*.

1.4 Struttura del documento

I capitoli successivi guidano il lettore attraverso le fasi di analisi, selezione tecnologica, progettazione, implementazione e valutazione del progetto, e sono organizzati come segue:

- **Capitolo 2 – Tecnologie e strumenti utilizzati:** descrive lo stack tecnologico adottato, giustificando le scelte chiave in relazione agli obiettivi del progetto e ai vincoli di sviluppo.
- **Capitolo 3 – Contesto e analisi del sistema:** definisce in dettaglio il problema, analizza lo stato dell'arte e presenta i requisiti del sistema.
- **Capitolo 4 – Progettazione e implementazione del sistema:** descrive l'architettura software, il design pattern adottato, la struttura del database e l'implementazione delle principali funzionalità, con esempi di codice e diagrammi tecnici.
- **Capitolo 5 – Distribuzione e prodotto finale:** presenta il processo di distribuzione, l'interfaccia utente e i flussi di utilizzo reali.
- **Capitolo 6 – Conclusioni:** riflette sui risultati ottenuti e sul percorso di crescita personale.

Tecnologie e strumenti utilizzati

La realizzazione del progetto **Swift Contest** è stata resa possibile dall'adozione di un insieme di tecnologie, piattaforme e strumenti moderni, selezionati per soddisfare i requisiti di cross-platform, sicurezza, scalabilità e velocità di sviluppo. Questo capitolo descrive lo stack tecnologico utilizzato, giustificando le scelte chiave in relazione agli obiettivi del progetto.

2.1 Framework, piattaforme e linguaggi

Questa sezione descrive le tecnologie fondamentali su cui si basa l'architettura software del progetto, partendo dal frontend fino ad arrivare ai linguaggi di programmazione utilizzati.

2.1.1 Framework Frontend



Flutter [1] è il framework UI open-source di Google scelto per lo sviluppo dell'applicazione client. La sua caratteristica principale è la capacità di generare applicazioni compilate nativamente per mobile, web e desktop da un'unica codebase. Questa scelta è stata fondamentale per soddisfare il requisito di **sviluppo cross-platform**, garantendo un'esperienza utente coerente e riducendo drasticamente i tempi di sviluppo e manutenzione.

2.1.2 Framework e piattaforme Backend



Supabase [2] è la piattaforma open-source **Backend-as-a-Service (BaaS)** che costituisce il cuore dell'infrastruttura server. La scelta è ricaduta su Supabase dopo un'attenta valutazione delle principali alternative, in particolare **Firestore** [5] di Google. Supabase è stato preferito per una ragione architeturale fondamentale: il suo backend è interamente basato su **PostgreSQL**.



PostgreSQL [6] è il potente sistema open-source per la gestione di database relazionali su cui si basa l'intera piattaforma Supabase. La decisione di adottare un backend basato su SQL, e in particolare su PostgreSQL, è stata un fattore determinante nella scelta tecnologica, in quanto garantisce integrità referenziale e la possibilità di eseguire query complesse, a differenza delle alternative basate su database NoSQL come Firebase.

2.1.3 Linguaggi di programmazione



Dart [7] è il linguaggio di programmazione su cui si basa il framework Flutter. Essendo una conseguenza diretta della scelta di Flutter, le sue caratteristiche si sono rivelate ideali per gli obiettivi del progetto: è un linguaggio moderno, orientato agli oggetti e fortemente tipizzato, la cui architettura supporta sia la compilazione *Just-In-Time (JIT)* per uno sviluppo rapido (con *hot reload*), sia la compilazione *Ahead-Of-Time (AOT)* per garantire performance elevate in produzione.



TypeScript [8] è il linguaggio richiesto dal runtime Deno per lo sviluppo delle **Edge Functions** su Supabase. Sebbene non sia stata una scelta attiva, la sua natura fortemente tipizzata, unita alla vasta disponibilità di librerie dell'ecosistema JavaScript, si è rivelata ideale per orchestrare le interazioni sicure con le API di servizi esterni.

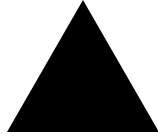


PL/pgSQL [6] è il linguaggio procedurale nativo di PostgreSQL. È stato utilizzato per scrivere tutte le **Funzioni RPC (Remote Procedure Call)** del sistema, una scelta che si è rivelata fondamentale per garantire l'atomicità delle operazioni complesse e per ottenere performance elevate.

2.2 Strumenti, servizi e piattaforme di supporto

Questa sezione descrive le piattaforme di terze parti e gli strumenti che hanno supportato il ciclo di vita del progetto.

2.2.1 Piattaforme di distribuzione e servizi Cloud



Vercel [9] è la piattaforma serverless utilizzata per la distribuzione e l'hosting della versione **Web** dell'applicazione Flutter. La scelta è ricaduta su Vercel principalmente per il suo **piano gratuito** e per una pipeline di **CI/CD** completamente automatizzata.



Resend [10] è il servizio transazionale di invio **email** utilizzato dal sistema, integrato tramite la propria API REST per gestire l'invio di tutte le email automatiche.



Google Cloud è stato utilizzato per uno dei suoi servizi API: la **Google Places API** [11]. Questo servizio è stato integrato per fornire la funzionalità di ricerca e autocompletamento degli indirizzi.

2.2.2 Strumenti di sviluppo e amministrazione



IntelliJ IDEA [12] è l'IDE di JetBrains utilizzato per la scrittura del codice. Si è scelto di migrare da **Android Studio** a **IntelliJ IDEA Community Edition** per una maggiore stabilità e reattività.



Node Package Manager (npm) [13] è stato uno strumento indispensabile per l'installazione e la gestione di dipendenze di sviluppo, prima tra tutte la **CLI di Supabase**.



Retool [14] è la piattaforma *low-code* utilizzata per costruire rapidamente la **dashboard di amministrazione** interna.

2.2.3 Version Control e Code Hosting



Git [15] è il sistema di **controllo di versione distribuito** utilizzato per tracciare ogni modifica al codice sorgente del progetto.



GitHub [16] è la piattaforma di **code hosting** che ospita il repository Git del progetto, integrata con la pipeline di CI/CD di Vercel.

2.2.4 Strumenti di documentazione e modellazione



PlantUML [17] è lo strumento open-source utilizzato per creare tutti i diagrammi UML e architetturali. La sua caratteristica distintiva è l’approccio “diagrams as code”: i diagrammi vengono descritti attraverso un **linguaggio DSL (Domain-Specific Language)** testuale.

Contesto e analisi del sistema

3.1 Il problema: frammentazione e inefficienza nella gestione dei contest

La gestione di un contest è un processo che coinvolge una molteplicità di attori (organizzatori, partecipanti, giurati) e fasi operative sequenziali e dipendenti. L'approccio tradizionale si affida a un assemblaggio di strumenti generici e non integrati, creando un flusso di lavoro *discontinuo, inefficiente e soggetto a errori*.

Le criticità di questo modello possono essere analizzate nel dettaglio:

- **Frammentazione del processo e dei dati:** la comunicazione degli inviti avviene tramite servizi di posta elettronica, la sottomissione dei lavori tramite posta elettronica o piattaforme di cloud storage (es. Google Drive) e la tabulazione dei voti tramite fogli di calcolo (es. Google Sheets). Questo approccio disperde le informazioni, rende difficile avere una visione d'insieme e richiede un costante lavoro manuale di organizzazione dei dati.
- **Mancanza di integrità e consistenza dei dati:** la gestione manuale aumenta esponenzialmente il rischio di errori. Esempi comuni includono la perdita di

iscrizioni, l'errata trascrizione dei punteggi o l'inclusione di partecipanti non idonei. Non esiste un'unica **"fonte di verità"** per i dati del contest.

- **Carenza di strumenti in tempo reale:** l'assenza di un sistema centralizzato impedisce di gestire sessioni di voto live in modo coordinato e trasparente.
- **Rischi di sicurezza e potenziale di abuso:** la sottomissione di voti multipli, la partecipazione di utenti non invitati e la gestione di conflitti di interesse diventano problemi concreti e di difficile gestione.

L'obiettivo primario del sistema **Swift Contest** è superare queste criticità strutturali, fornendo un ambiente *end-to-end* integrato, sicuro e automatizzato, che funga da unica fonte di verità per l'intero ciclo di vita del contest.

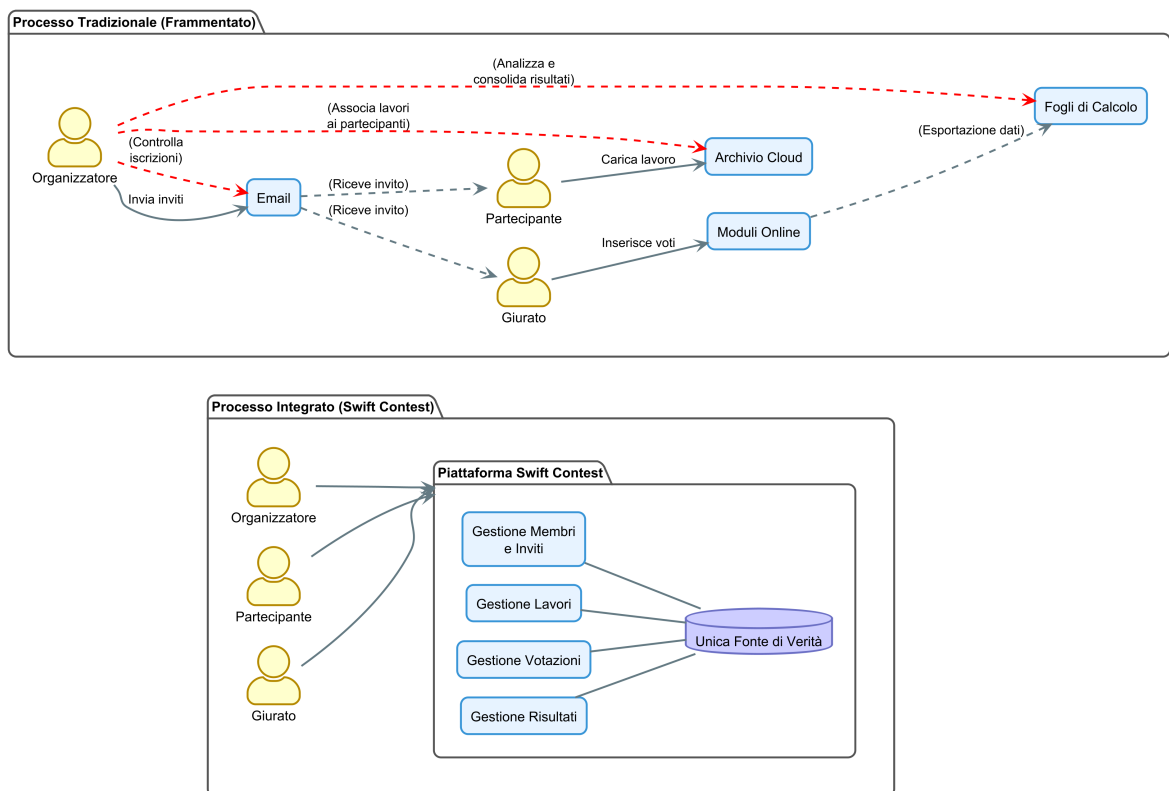


Figura 3.1: Confronto tra il flusso di lavoro tradizionale e l'approccio integrato di Swift Contest.

3.2 Stato dell'arte e analisi delle alternative

L'analisi dello stato dell'arte ha rivelato un'interessante lacuna nel panorama software attuale: **non esistono soluzioni *verticali* e accessibili che coprano in modo integrato l'intero ciclo di vita di un contest**. Le pratiche correnti si affidano a due approcci principali, entrambi inadeguati.

3.2.1 Strumenti generici adattati: lo "Status Quo"

Questa è la pratica più diffusa e rappresenta il problema fondamentale che **Swift Contest** si propone di risolvere. Consiste nell'utilizzare una combinazione di strumenti generici, non pensati per questo scopo, creando un flusso di lavoro frammentato e di difficile gestione, ad esempio:

- **Google Forms** per la raccolta delle iscrizioni e la sottomissione dei voti.
- **Google Drive** per la sottomissione dei lavori.
- **Google Sheets** per la tabulazione manuale dei voti.
- **Email** per tutte le comunicazioni.

Come già analizzato, questo approccio soffre di gravi criticità in termini di sicurezza, integrità dei dati ed efficienza operativa, che hanno motivato la nascita del progetto.

3.2.2 Piattaforme per domini correlati

Sono state analizzate anche piattaforme software specializzate in domini vicini a quello dei contest, per verificare la possibilità di un loro adattamento. Tuttavia, nessuna di esse si è rivelata una soluzione praticabile.

Piattaforme di e-voting

Sistemi come *Helios Voting* [3] sono progettati per il **voto politico o assembleare**. Il loro focus è sulla crittografia e sull'anonimato del votante. Di conseguenza, mancano completamente delle funzionalità necessarie per un contest, quali:

- La possibilità di invitare partecipanti e creare giurie.

- La sottomissione e gestione di opere digitali.
- La creazione di form di voto personalizzati con campi testuali e numerici.

Il loro scopo è intrinsecamente diverso e non sono in alcun modo utilizzabili.

Piattaforme per la gestione di eventi

Soluzioni commerciali come *Eventbrite* [4] sono focalizzate sulla **gestione logistica di eventi e sulla vendita di biglietti**. Sebbene eccellano nella gestione delle registrazioni, non offrono alcuna funzionalità specifica per il dominio dei contest, come:

- La sottomissione di opere digitali.
- La creazione di giurie con ruoli definiti.
- La configurazione di form di valutazione strutturati.
- La gestione di sessioni di voto in tempo reale.

Il loro scopo è completamente diverso e non risultano realisticamente adattabili ai requisiti di un contest.

3.2.3 Posizionamento di Swift Contest

L'analisi conferma che Swift Contest si posiziona in un **vuoto di mercato e tecnologico**. Non mira a competere con le piattaforme di e-voting o di gestione eventi, ma a creare una **nuova categoria di prodotto**: una soluzione verticale, integrata e accessibile, progettata specificamente per le esigenze di organizzatori, partecipanti e giurati di un contest.

3.3 Attori e casi d'uso

Il sistema è progettato per servire diversi tipi di utenti, ciascuno con un ruolo e un insieme di permessi ben definiti. L'interazione tra questi attori e le funzionalità del sistema è stata modellata attraverso una serie di diagrammi dei Casi d'Uso,

suddivisi in pacchetti logici (packages) che legano funzionalità della stessa categoria per migliorarne leggibilità e modularità.

3.3.1 Attori del sistema

L'analisi del dominio ha permesso di identificare i seguenti attori principali:

- **Ospite:** rappresenta un utente non autenticato. In questo sistema, che non prevede contenuti pubblici, un *Ospite* non ha privilegi di visualizzazione. Le sue uniche capacità sono quelle di *autenticarsi* (tramite registrazione o login) o di *agire come Giurato Semplice* se in possesso di un token di accesso.
- **Utente Autenticato:** è un attore astratto che rappresenta un utente che ha completato l'autenticazione. Funge da base per i ruoli operativi, che ne ereditano le funzionalità comuni come la gestione del proprio profilo e delle notifiche.
- **Organizzatore:** è un utente autenticato responsabile della creazione e della gestione completa di un contest.
- **Partecipante:** è un utente autenticato che si iscrive a un contest per sottoporre il proprio lavoro.
- **Giurato:** è un utente autenticato il cui ruolo primario è quello di far parte di una giuria nominata, oppure votare come Giurato Semplice se in possesso di un token per la votazione.
- **Amministratore:** è un ruolo di amministrazione, gestito tramite una dashboard esterna (Retool), con privilegi elevati per la manutenzione del sistema.

3.3.2 Casi d'Uso: gestione account e funzionalità generali

Questa area copre le funzionalità di base legate all'accesso e alla gestione dell'account utente. Include i processi di autenticazione, la modifica del profilo personale e la gestione delle notifiche interne.

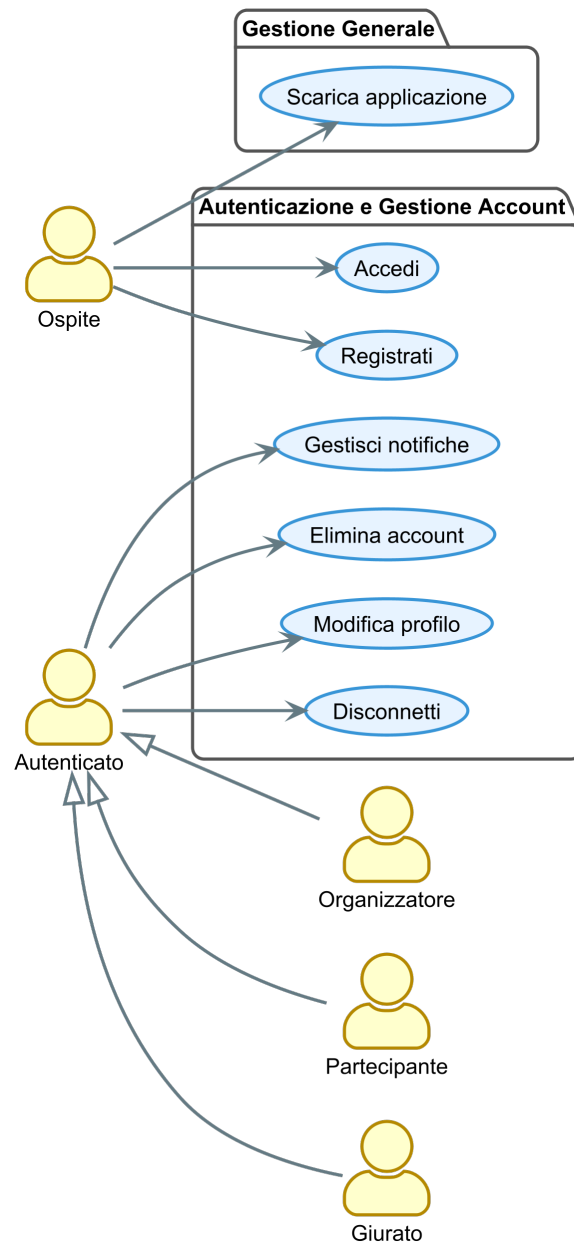


Figura 3.2: Diagramma dei casi d'uso: gestione account e funzionalità generali.

3.3.3 Casi d'Uso: gestione del ciclo di vita del contest

Questa è l'area funzionale più vasta e descrive il cuore del processo organizzativo. Copre la creazione di un contest, la gestione dei suoi membri (partecipanti e giurati) e la sottomissione dei lavori. Gli attori principali in questo contesto sono l'*Organizzatore* e il *Partecipante*.

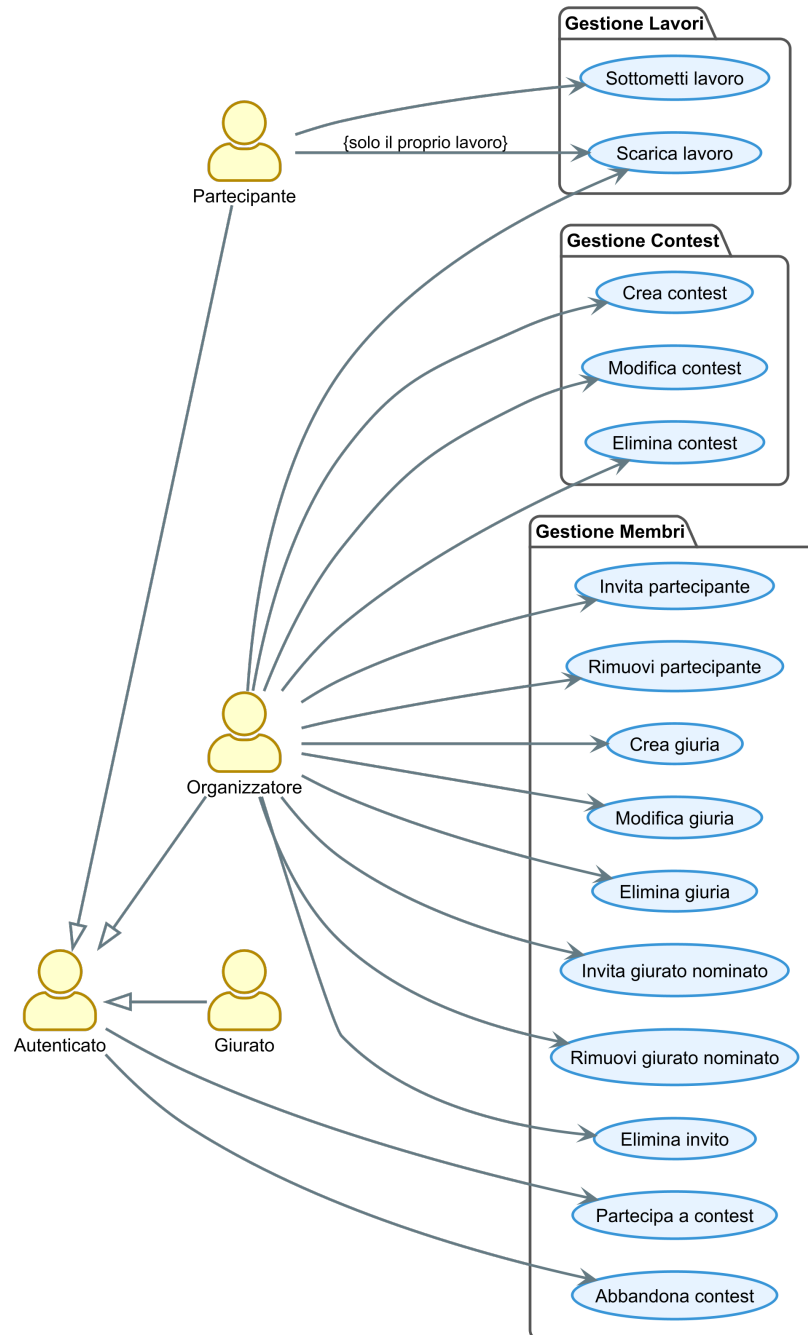


Figura 3.3: Diagramma dei casi d'uso: gestione del ciclo di vita del contest.

3.3.4 Casi d'Uso: gestione delle votazioni e dei risultati

Questa sezione si concentra interamente sul processo critico della votazione. Include la configurazione del form di voto, la conduzione della sessione di voto (anche con restrizioni di geolocalizzazione) e la generazione e pubblicazione delle classifiche. Gli attori principali qui sono l'*Organizzatore* e il *Giurato*.

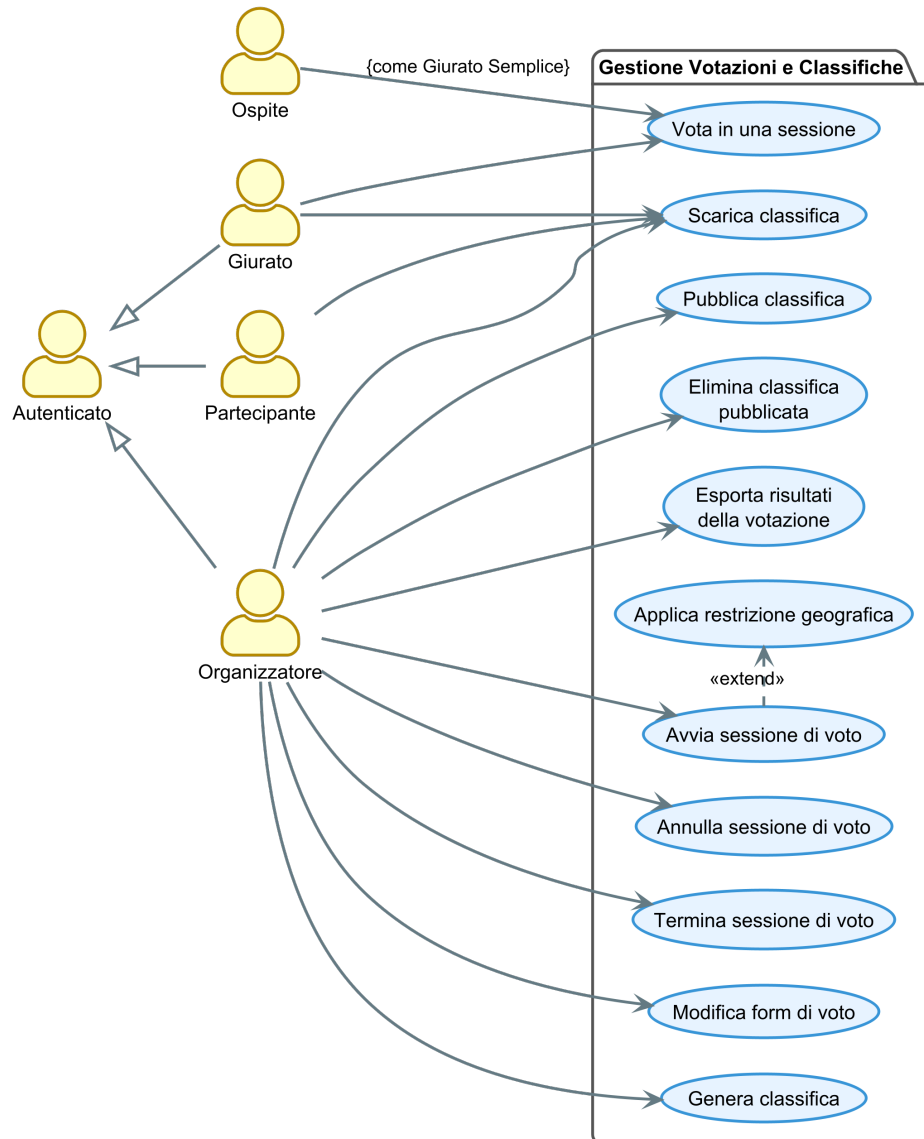


Figura 3.4: Diagramma dei casi d'uso: gestione delle votazioni e dei risultati.

3.3.5 Casi d'Uso: amministrazione del sistema

Quest'ultima area isola le funzionalità di amministrazione, che non sono accessibili tramite l'applicazione principale ma attraverso una dashboard esterna. Queste operazioni sono riservate all'attore *Amministratore* e sono fondamentali per la manutenzione e la moderazione della piattaforma.

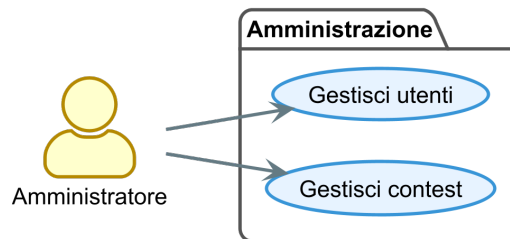


Figura 3.5: Diagramma dei casi d'uso: amministrazione del sistema.

3.4 Requisiti del sistema

I requisiti del sistema sono stati classificati secondo il modello **FURPS+**, un'estensione del modello FURPS che include categorie aggiuntive per coprire in modo esaustivo tutti i vincoli di un progetto software.

3.4.1 Requisiti funzionali (Functionality)

- **RF-01: Gestione Account Utente:** Il sistema deve permettere a un utente di registrarsi e autenticarsi.
- **RF-02: Gestione del Profilo Personale:** Ogni utente autenticato deve poter visualizzare e modificare i dati del proprio profilo (es. nome completo). Deve inoltre essere garantita la possibilità di eliminare il proprio account in modo autonomo.
- **RF-03: Gestione Contest:** Il sistema deve permettere a un *Organizzatore* di creare, modificare e cancellare i propri contest.

- **RF-04: Gestione Giurie e Form di Voto:** Il sistema deve permettere a un *Organizzatore* di creare giurie (di tipo *nominata* o *semplice*) e di configurare un form di votazione associato.
- **RF-05: Gestione Iscrizioni:** Il sistema deve permettere a un *Organizzatore* di invitare partecipanti e giurati nominati tramite email con token univoci.
- **RF-06: Gestione Esclusioni di Voto:** Il sistema deve permettere a un *Organizzatore* di configurare delle esclusioni, impedendo a specifici *Giurati Nominati* di valutare determinati *Partecipanti* all'interno di una sessione di voto.
- **RF-07: Accettazione Inviti e Accesso al contest:** Il sistema deve permettere a un utente di utilizzare un token di invito per formalizzare la propria iscrizione a un contest, sia come *Partecipante* che come *Giurato Nominato*.
- **RF-08: Sottomissione Lavori:** Il sistema deve permettere a un *Partecipante* di sottomettere un proprio lavoro per un contest a cui è iscritto.
- **RF-09: Restrizione Geografica della Votazione:** Il sistema deve permettere a un *Organizzatore* di imporre una restrizione geografica a una sessione di voto, richiedendo ai giurati di trovarsi entro un raggio specifico da un luogo predefinito.
- **RF-10: Processo di Votazione:** Il sistema deve permettere a un utente autorizzato (sia *Giurato Nominato* che *Giurato Semplice*) di inviare i propri voti durante una sessione di voto attiva, rispettando le eventuali esclusioni configurate.
- **RF-11: Gestione Classifiche e Risultati:** Il sistema deve permettere all'*Organizzatore* di generare una classifica per ogni singola giuria, di esportare i risultati grezzi in formato Excel e di pubblicare una classifica finale in formato PDF.
- **RF-12: Visualizzazione dei Risultati:** Il sistema deve permettere a *Partecipanti* e *Giurati* di un contest di visualizzare e scaricare le classifiche finali pubblicate dall'organizzatore.

- **RF-13: Notifiche Inbox:** Il sistema deve fornire a ogni utente autenticato una inbox per ricevere notifiche generate automaticamente da eventi di sistema.
- **RF-14: Amministrazione del Sistema:** Il sistema deve esporre un'interfaccia sicura per permettere a un *Amministratore* di eseguire operazioni di alto livello, come la visualizzazione di tutti gli utenti o l'eliminazione forzata di un contest.

3.4.2 Requisiti non funzionali

- **Usabilità (Usability):** L'interfaccia utente deve essere intuitiva, facile da apprendere e coerente tra la versione mobile e quella web.
- **Affidabilità (Reliability):** Il sistema deve essere robusto e garantire l'integrità dei dati. La logica di *snapshot* delle sessioni di voto è un requisito chiave per assicurare che il processo di valutazione sia immune da modifiche ai dati.
- **Performance:** L'applicazione deve essere reattiva, con tempi di caricamento inferiori ai 3 secondi.
- **Manutenibilità (Supportability):** L'architettura software deve essere modulare. L'adozione del pattern MVVM/BLoC per il frontend garantisce una netta separazione delle responsabilità, facilitando futuri interventi.
- **Implementazione (Implementation):**
 - Il client deve essere sviluppato in **Dart** con il framework **Flutter** per le piattaforme Android e Web.
 - Il backend deve basarsi sulla piattaforma **Supabase**, utilizzando **PostgreSQL**, **PL/pgSQL** per le RPC e **TypeScript** per le Edge Functions.
 - La gestione dello stato nel client deve seguire il pattern **BLoC**.
- **Interfaccia (Interface):**
 - Il sistema deve interfacciarsi in modo sicuro con servizi esterni come **Resend** (per le email) e **Google Places** (per la geolocalizzazione).

- Deve essere prevista un'API sicura per l'interfacciamento con strumenti di amministrazione esterni (**Retool**).
- **Operativi (Operation):**
 - La distribuzione del backend deve essere tracciabile tramite migrazioni SQL gestite dalla CLI di Supabase.
 - La distribuzione del frontend web deve essere automatizzata tramite una pipeline di **CI/CD**.
 - I segreti (API keys) devono essere gestiti in modo sicuro, senza essere esposti nel codice sorgente.
- **Distribuzione (Packaging):**
 - L'applicazione web deve essere accessibile tramite URL pubblico.
 - L'applicazione Android deve essere distribuita come pacchetto **APK**, tramite un meccanismo di download sicuro.
- **Legali (Legal):**
 - Il sistema deve essere sviluppato utilizzando principalmente tecnologie con licenze open-source permissive.
 - La gestione dei dati personali deve essere orientata ai principi del GDPR, che garantisce all'utente il diritto alla cancellazione.

[illegible]

22

3.5.1 Entità fondamentali

- **User:** l'utente del sistema, contenente informazioni del profilo.
- **Contest:** l'entità centrale, che aggrega tutte le informazioni relative a un concorso.
- **ParticipantMembership** e **JurorMembership:** *classi associative* che risolvono le relazioni “multi-a-molti” tra utenti e contest/giurie.
- **Work:** l'elaborato unico caricato da un partecipante. È legato a una *ParticipantMembership*.
- **Jury:** una giuria all'interno di un contest, che può essere di tipo *nominata* o *semplice*.
- **VotingForm** e **VotingFormField:** la struttura del form di votazione. L'attributo *scope* del campo ne determina il comportamento.
- **VotingSession:** una sessione di voto live. Al suo avvio, il sistema “conge-la” lo stato del contest creando delle righe dedicate in tabelle di sessione (es. `voting_session_participants`), che contengono non solo i riferimenti alle entità originali, ma anche una copia di **attributi di snapshot** (es. il nome del partecipante, il nome del lavoro) per garantire l'immutabilità del processo.
- **ContestRanking:** la classifica finale, rappresentata come un riferimento a un file nello storage.
- **Message:** una notifica di sistema inviata a un utente.

3.5.2 Vincoli chiave del dominio

Il modello dati è stato progettato per far rispettare programmaticamente alcune regole di business fondamentali:

1. **Unicità del lavoro:** un partecipante può sottomettere un solo lavoro per ogni concorso a cui è iscritto, garantito da un *vincolo di unicità* a livello di database.

2. **Calcolo della Classifica Semplice:** la classifica viene generata esclusivamente sulla base dei campi di tipo *slider* con scope *participant*. I campi testuali non contribuiscono al punteggio.
3. **Immutabilità della Votazione:** il **meccanismo di snapshot** garantisce che una volta avviata una sessione di voto, i dati rilevanti dei partecipanti, dei giurati e la struttura del form vengano “congelati”. Questo avviene tramite la copia di attributi chiave in tabelle di sessione dedicate, rendendo il processo di valutazione immune da modifiche successive ai dati originali.
4. **Accesso Ospite Controllato:** l’accesso come *Giurato Semplice* è strettamente controllato da un token associato a una specifica giuria di tipo semplice.

Progettazione e implementazione del sistema

4.1 Linee guida progettuali

La progettazione di **Swift Contest** è stata guidata da principi di ingegneria del software moderni, volti a garantire un sistema robusto, sicuro e manutenibile, anche in un contesto di sviluppo rapido. Le linee guida principali sono state:

- **Separazione delle responsabilità (SoC)**: è stata applicata una netta separazione tra il livello di presentazione (UI), la logica di presentazione (ViewModel) e il livello di accesso ai dati (Model/Repository), fondamentale per la manutenibilità e scalabilità del codice.
- **Sicurezza come requisito primario**: è stato adottato un modello di accesso ai dati basato sul **Principio del Minimo Privilegio (Principle of Least Privilege)**, dove il client è considerato intrinsecamente non sicuro. L'accesso al database è negato per default e concesso selettivamente solo tramite un'API sicura.
- **Architettura Client-Server con backend BaaS**: è stata adottata una moderna architettura Client-Server. Il client (sviluppato in Flutter) gestisce la presentazione e la logica di UI, mentre il server è rappresentato da una piattaforma **Backend-as-a-Service (BaaS)** come Supabase. Questo approccio permette di

delegare la gestione dell'infrastruttura (database, autenticazione, storage) e di distribuire la logica di business in modo flessibile tra il client e le funzioni serverless (RPC ed Edge Functions).

- **Sviluppo cross-platform:** la scelta di un framework come Flutter ha permesso di massimizzare il riutilizzo del codice tra le piattaforme target (Android e Web), ottimizzando i tempi di sviluppo.

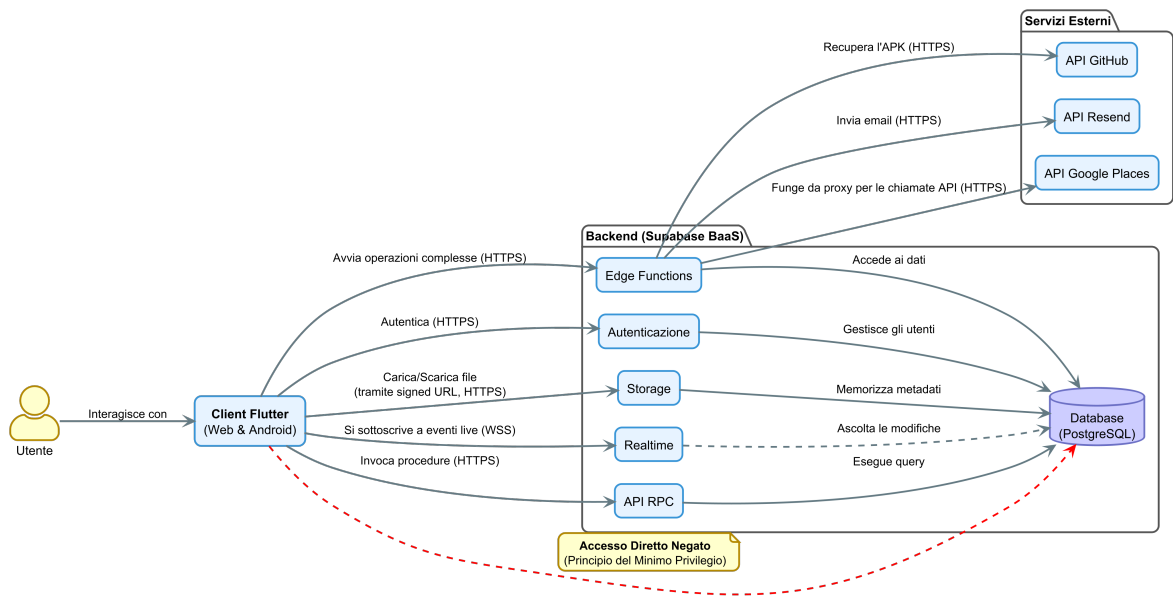


Figura 4.1: Architettura Client-Server del sistema Swift Contest con backend BaaS.

4.2 Architettura del frontend: il pattern MVVM con BLoC

Il frontend è stato sviluppato in Flutter seguendo i principi dell'architettura **MVVM (Model-View-ViewModel)**, implementata attraverso il pattern **BLoC (Business Logic Component)**. Questa scelta permette di ottenere una netta separazione delle responsabilità, garantendo non solo **testabilità** e **manutenibilità**, ma anche la possibilità di **sviluppare la UI e la logica di business in parallelo**. La struttura del codice sorgente in `lib/` riflette questa suddivisione:

- **View (/view):** il livello di presentazione, composto dai Widget di Flutter che costituiscono l'interfaccia utente. Questo livello si limita a mostrare i dati ricevuti

ti dal suo ViewModel (il BLoC) e a notificare gli eventi di interazione dell'utente (es. la pressione di un pulsante).

- **ViewModel (/viewmodel, implementato come BLoC):** il cuore della logica di presentazione. Ogni BLoC riceve eventi dalla View, esegue la logica di business (spesso invocando un metodo del Model/Repository), e produce uno stream di stati a cui la View reagisce per aggiornarsi.
- **Model (/model):** il livello di accesso ai dati. È composto dai *Repository*, che fungono da unica interfaccia verso il backend, astruendo le chiamate a RPC ed Edge Functions. Include anche le *Entities* (le rappresentazioni dei dati del dominio) e i *Bundles*, ovvero oggetti aggregati che raggruppano dati correlati (es. un contest e i suoi partecipanti) in un'unica struttura, **ottimizzando il numero di chiamate di rete necessarie** per popolare una schermata.

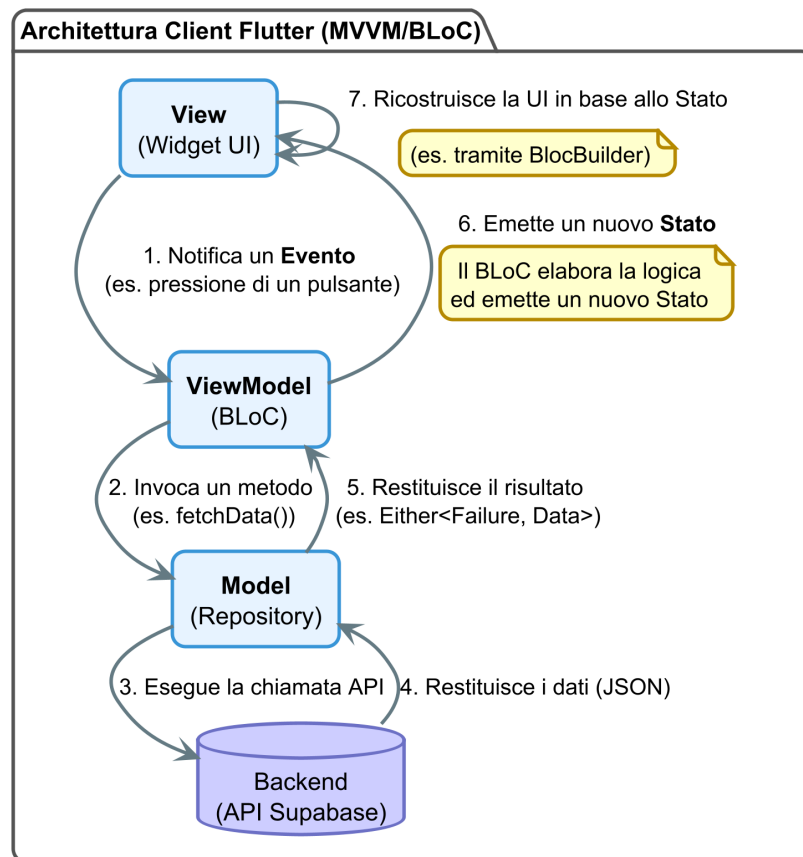


Figura 4.2: Architettura del Client MVVM con BLoC.

4.2.1 Implementazione del ViewModel: il pattern BLoC

Per la gestione dello stato e l'implementazione del ViewModel, è stato adottato il pattern BLoC, utilizzando il pacchetto `flutter_bloc` [18]. La scelta di BLoC è motivata dalla sua natura reattiva e basata su stream, che si integra perfettamente con il paradigma dichiarativo di Flutter. Questo pattern impone un flusso di dati unidirezionale e prevedibile:

1. La **View** notifica un *Evento* (es. `LoginButtonPressed`).
2. Il **BLoC** riceve l'evento, esegue la logica di business (es. chiama `authRepository.login()`) e, al termine, emette un nuovo *Stato* (es. `LoginSuccess` o `LoginFailure`).
3. La **View**, tramite un widget come `BlocBuilder`, si mette in ascolto dello stream di stati e si ridisegna per riflettere il nuovo stato.

Questo approccio garantisce che la logica di business sia completamente disaccoppiata dalla UI, rendendola facilmente testabile in isolamento. I BLoC sono stati organizzati in due categorie: **globali** (es. `AuthBloc`, `InboxBloc`) e **di pagina** (es. `JurorVotingProcedurePageBloc`).

Gestione centralizzata degli errori nel repository

Una delle responsabilità principali del livello *Model* (e in particolare dei *Repository*) è quella di agire come un robusto intermediario tra la logica di business e il backend. Questo include non solo il recupero dei dati, ma anche una **gestione centralizzata e sicura degli errori**.

Le chiamate a un backend come Supabase possono fallire in molti modi: problemi di rete (`SocketException`), errori di autenticazione (`AuthException`), errori del database (`PostgreSQLException`), ecc. Esporre queste eccezioni direttamente al livello del BLoC o della UI sarebbe problematico per due motivi:

1. Creerebbe un forte accoppiamento tra la logica di presentazione e le librerie specifiche del backend.

2. Potrebbe esporre all'utente finale messaggi di errore tecnici che, oltre a essere incomprensibili, rappresentano un **rischio per la sicurezza** (information leakage), rivelando dettagli sull'implementazione del database o del backend.

Per risolvere questo problema, è stata creata una funzione di ordine superiore, `handleBackendCall`, che “avvolge” ogni chiamata al backend. Come mostrato nel listato seguente, questa funzione utilizza un blocco `try...catch` per intercettare le eccezioni specifiche di Supabase e mapparle in classi di **Failure** personalizzate e sicure. Queste *Failure* sono oggetti semplici che rappresentano un fallimento in modo astratto (es. `NetworkFailure`, `ServerFailure`), senza contenere dettagli tecnici.

Il risultato di questa mappatura viene quindi incapsulato utilizzando il tipo `Either`, fornito dal pacchetto di programmazione funzionale `fpdart` [19]. **Either** è un tipo di dato che può contenere solo uno di due valori possibili: un valore di “sinistra” (*Left*), che per convenzione rappresenta il fallimento, o un valore di “destra” (*Right*), che rappresenta il successo.

Il tipo di ritorno `Either<Failure, T>` formalizza quindi questo contratto: la funzione restituirà o un oggetto di tipo `Failure` (a sinistra) o il dato atteso di tipo `T` (a destra). Questo approccio elimina la necessità per il codice chiamante (il BLoC) di usare blocchi `try...catch`, permettendogli di gestire in modo pulito, dichiarativo e fortemente tipizzato sia il caso di successo che quello di fallimento, semplicemente ispezionando il contenuto di `Either`.

```

1 Future<Either<Failure, T>> handleBackendCall<T>(
2   Future<Either<Failure, T>> Function() function,
3 ) async {
4   try {
5     return await function();
6   } on SocketException catch (e) {
7     // Errore di rete (es. no internet).
8     Logger.warning('Network error: $e');
9     return Either.left(const NetworkFailure());
10  } on PostgreSQLException catch (e) {
11    // Errore del database PostgreSQL.
12    Logger.warning('Postgrester error: $e');
13    return Either.left(postgresterExceptionToFailure(e));
14  }
15  // ... altri catch specifici per le eccezioni di Supabase ...
16  catch (e) {

```

```
17 // Qualsiasi altro errore non previsto.  
18 Logger.error('Unknown error: $e');  
19 return Either.left(const Failure());  
20 }  
21 }
```

Listato 4.1: Funzione helper per la gestione sicura delle chiamate al backend.

Gestione dell'uguaglianza degli oggetti con Equatable

Per ottimizzare le performance e garantire un comportamento prevedibile nella gestione dello stato, tutte le entità, i bundle e gli stati dei BLoC estendono la classe fornita dal pacchetto `equatable` [20].

Questa scelta implementativa è cruciale per due motivi principali.

Il primo riguarda il corretto funzionamento del pattern BLoC. Di default, Dart confronta gli oggetti per riferimento, non per valore. Ciò significa che due istanze di un oggetto, anche se contengono dati identici, vengono considerate diverse. Senza `Equatable`, un BLoC potrebbe emettere un nuovo stato identico al precedente, causando una ricostruzione non necessaria e dispendiosa dell'interfaccia utente. Utilizzando `Equatable`, invece, il confronto avviene basandosi sul valore degli attributi dell'oggetto. Questo permette ai widget di Flutter, come `BlocBuilder`, di determinare in modo efficiente se uno stato è **realmente** cambiato prima di procedere con la ricostruzione, prevenendo refresh superflui della UI e migliorando significativamente la reattività dell'applicazione.

Il secondo motivo, altrettanto importante, riguarda la *robustezza e la chiarezza della logica di business*. Spesso, durante lo sviluppo, sorge la necessità di confrontare due oggetti per verificare se rappresentano la stessa entità concettuale, anche se sono istanze diverse in memoria. Estendendo `Equatable`, si ottiene un'implementazione standard e priva di errori dell'uguaglianza basata sul valore, semplificando la logica di confronto all'interno dei BLoC e dei Repository e riducendo il rischio di bug sottili legati al confronto per riferimento.

4.2.2 Navigazione con AutoRoute

La gestione della navigazione è stata affidata al pacchetto `auto_route` [21], che genera un router fortemente tipizzato, eliminando l'uso di stringhe per le rotte. Questo approccio trasforma potenziali errori di runtime (es. un errore di battitura nel nome di una rotta) in **errori di compilazione**, aumentando significativamente la robustezza dell'applicazione. Un elemento chiave è l'`AuthGuard`, una classe che intercetta le richieste di navigazione e verifica lo stato di autenticazione dell'utente tramite l'`AuthBloc`, reindirizzandolo alla pagina di login se tenta di accedere a una rotta protetta.

```

1 // --- 1. Definizione della rotta protetta (in app_router.dart) ---
2 AutoRoute(
3   path: '/organizer-home',
4   page: OrganizerHomeRoute.page,
5   guards: [AuthGuard(authBloc: authBloc)], // Applica la guardia
6 ),
7
8 // ...
9
10 // --- 2. Implementazione della guardia (in auth_guard.dart) ---
11 class AuthGuard extends AutoRouteGuard {
12   final AuthBloc authBloc;
13   AuthGuard({required this.authBloc});
14
15   @override
16   void onNavigation(NavigationResolver resolver, StackRouter router) {
17     // Controlla lo stato di autenticazione tramite l'AuthBloc
18     final isAuthenticated =
19       ⇨ authBloc.state.authStatus.isAuthenticated;
20
21     // Logica di base: se l'utente non è autenticato,
22     // viene reindirizzato alla pagina di login.
23     if (isAuthenticated) {
24       resolver.next(true); // Procede alla rotta protetta
25     } else {
26       resolver.redirect(SignInRoute()); // Reindirizza al login
27     }
28   }
29 }

```

Listato 4.2: Estratto del sistema di routing protetto con “AuthGuard”.

4.2.3 Strategia di navigazione e passaggio dei dati

Una scelta architetturale fondamentale, dettata dalla necessità di una perfetta integrazione con il web, riguarda la modalità di passaggio dei dati tra le pagine. A differenza di un'applicazione puramente mobile, dove è comune passare interi oggetti in memoria durante la navigazione, in un'applicazione web questo approccio è insostenibile: l'utente può ricaricare la pagina o accedere a un URL diretto, perdendo qualsiasi stato tenuto in memoria.

Per questo motivo, il sistema di navigazione di Swift Contest adotta una strategia rigorosa: **nelle rotte vengono passati solo identificativi (ID) primitivi** (es. `contestId`, `juryId`), mai oggetti complessi. Quando una pagina viene caricata, il BLoC associato riceve l'ID dai parametri della rotta e, come prima azione, invoca il Repository per caricare dal backend tutti i dati necessari.

Questa scelta, sebbene aumenti il numero di chiamate al database, garantisce vantaggi in termini di robustezza e coerenza con il paradigma web:

- **Ogni pagina è ricaricabile** in modo indipendente.
- **Ogni pagina ha un URL univoco** e condivisibile (deep linking).
- **Il comportamento dell'applicazione è prevedibile** e consistente tra la navigazione interna e l'accesso diretto a un URL.

Validazione dell'input utente e gestione dei form

Per garantire l'integrità dei dati e fornire un feedback immediato all'utente, è stata implementata una robusta strategia di validazione dell'input direttamente a livello client, prima che i dati vengano inviati al BLoC e, successivamente, al backend.

Invece di definire la logica di validazione all'interno dei singoli widget, **tutte le funzioni di validazione sono state centralizzate** in un unico file, `lib/utils/validators/validators.dart`. Questo approccio, basato su funzioni pure, offre due vantaggi significativi:

- **Riutilizzabilità**: La stessa funzione, ad esempio `emailValidator`, può essere riutilizzata in più punti dell'applicazione (es. pagina di login, registrazione) senza duplicazione di codice.

- **Manutenibilità:** Se una regola di validazione deve essere modificata (es. la complessità minima di una password), la modifica viene effettuata in un unico punto, garantendo coerenza in tutta l'applicazione.

Il listato seguente mostra un esempio di un *validator* e il suo utilizzo all'interno di un widget `TextFormField`.

```
1 // 1. Definizione della funzione di validazione pura
2 String? passwordValidator(String? value) {
3     if (value == null || value.isEmpty) {
4         return 'Required';
5     }
6     if (value.length < 8) {
7         return 'Must contain at least 8 characters';
8     }
9     // ... altre regole di validazione (maiuscole, numeri, etc.) ...
10
11     return null; // La validazione ha successo
12 }
13
14 // 2. Utilizzo del validatore in un widget della UI
15 TextFormField(
16     validator: passwordValidator,
17     autovalidateMode: AutovalidateMode.onUserInteraction,
18     // ...
19 );
```

Listato 4.3: Esempio di validatore centralizzato e suo utilizzo in un widget.

4.2.4 Design system e theming

Per garantire un'interfaccia utente coerente e facilmente manutenibile, è stato definito un design system centralizzato basato sul **Material Design 3** [22]. La scelta di `MaterialTheme` come base è stata strategica, in quanto è lo standard de facto per le applicazioni Android e garantisce un'esperienza utente nativa e riconoscibile.

Il cuore del sistema di theming di Material Design è il **ColorScheme**, un oggetto che definisce una palette di colori predefinita e semanticamente organizzata. Invece di usare colori specifici i widget fanno riferimento a ruoli di colore come `primary` (per elementi principali), `secondary` (per elementi di supporto), `surface` (per

gli sfondi), `onPrimary` (per il testo da mostrare sopra un colore primario), ecc. Questo approccio permette di cambiare l'intera palette di colori dell'applicazione (ad esempio, passando dal tema chiaro a quello scuro) modificando unicamente l'oggetto `ColorScheme`, senza dover alterare il codice dei singoli widget.

Tuttavia, il `ColorScheme` standard non è sufficiente per coprire tutte le esigenze di un'applicazione complessa, che richiede colori semantici personalizzati (es. per stati di successo, errore, warning).

Per risolvere questo problema in modo pulito e scalabile, invece di definire colori statici sparsi nel codice, si è fatto ricorso a una funzionalità del linguaggio Dart: le **estensioni**. Un'estensione (*extension*) permette di **aggiungere nuove funzionalità a una classe esistente**, anche se non si ha accesso al suo codice sorgente. In questo caso, è stata creata un'estensione sulla classe `ColorScheme` di Flutter per "arricchirla" con i colori semantici personalizzati richiesti dall'applicazione.

Come mostrato nel listato seguente, l'estensione `ColorSchemeX` aggiunge nuovi getter (es. `success`, `warning`) alla classe `ColorScheme`. Questi nuovi colori diventano così parte integrante del tema. Per gestire l'adattamento tra tema chiaro e scuro, ogni getter personalizzato accede alla proprietà `brightness` della `ColorScheme` e, tramite un'espressione condizionale, restituisce il valore cromatico appropriato. In questo modo, un singolo richiamo come `Theme.of(context).colorScheme.success` restituisce il colore corretto sia in modalità chiara che scura, garantendo coerenza visiva in tutta l'applicazione.

```
1 // in lib/utils/themes/color_scheme_x.dart
2
3 extension ColorSchemeX on ColorScheme {
4     // Colore per indicare successo (es. notifiche positive)
5     Color get success => brightness == Brightness.light
6         ? const Color(0xFF28a745) // Verde brillante per tema chiaro
7         : const Color(0xFF218838); // Verde più scuro per tema scuro
8
9     Color get onSuccess => const Color(0xFFFFFFFF);
10
11     // Colore per avvisi
12     Color get warning => brightness == Brightness.light
13         ? const Color(0xFFffc107) // Giallo per tema chiaro
14         : const Color(0xFFd39e00); // Giallo ocra per tema scuro
15 }
```

```
16   Color get onWarning => const Color(0xFF212529);  
17  
18   // Altri colori personalizzati...  
19 }
```

Listato 4.4: Definizione dell'estensione "ColorSchemeX" per aggiungere colori personalizzati al tema.

Gestione del feedback di caricamento: l'OverlayLoader

Per fornire un feedback visivo all'utente durante le operazioni asincrone, è stato implementato un sistema di caricamento personalizzato, `OverlayLoader`. Questo sistema utilizza l'Overlay di Flutter per mostrare un indicatore di progresso con uno sfondo semitrasparente e offuscato, permettendo all'utente di mantenere il contesto visivo della schermata sottostante.

L'implementazione si basa su due scelte di design chiave per garantire robustezza e facilità d'uso:

- **Singleton Pattern:** La classe `OverlayLoader` è implementata come un singleton per garantire che esista una sola istanza del gestore del loader, prevenendo la visualizzazione di più overlay contemporaneamente.
- **Dart Extensions:** Per semplificare l'invocazione del loader, è stata creata un'estensione sulla classe `BuildContext`, che permette di usare una sintassi pulita e intuitiva come `context.showLoader()` e `context.hideLoader()`.

Gestione del ciclo di vita e prevenzione dei "leak" Un aspetto cruciale per la robustezza di questo sistema è la gestione del suo ciclo di vita. Per evitare che l'overlay del loader rimanga "bloccato" sullo schermo se una pagina viene chiusa (*disposed*) durante un'operazione di caricamento (ad esempio, a causa di un errore o della navigazione dell'utente), è stata adottata una pratica difensiva: **il metodo `context.hideLoader()` viene sistematicamente invocato all'interno del metodo `dispose` di ogni pagina** che utilizza il loader. Questo garantisce che, indipendentemente da come o perché la pagina viene distrutta, qualsiasi overlay di caricamento

attivo venga sempre rimosso, prevenendo “leak” visuali e garantendo la stabilità dell’interfaccia utente.

```

1 // 1. Classe Singleton per la gestione dell'overlay
2 class OverlayLoader {
3     OverlayEntry? _entry;
4
5     // Singleton pattern per garantire un'unica istanza
6     OverlayLoader._();
7     static final OverlayLoader _instance = OverlayLoader._();
8     factory OverlayLoader() => _instance;
9
10    void show(BuildContext context) {
11        if (_entry != null) return; // Evita di mostrare più loader
12
13        _entry = OverlayEntry(
14            builder: (_) => const Positioned.fill(
15                // ObscuredLoader è un widget custom con sfondo oscurato
16                child: ObscuredLoader(),
17            ),
18        );
19
20        // Inserisce l'entry nell'overlay radice per essere sempre in
21        // ↪ cima
22        Overlay.of(context, rootOverlay: true).insert(_entry!);
23    }
24
25    void hide() {
26        _entry?.remove();
27        _entry = null;
28    }
29
30    // 2. Estensione su BuildContext per un'API pulita e intuitiva
31    extension OverlayLoaderX on BuildContext {
32        void showLoader() => OverlayLoader().show(this);
33        void hideLoader() => OverlayLoader().hide();
34    }

```

Listato 4.5: Implementazione del loader non-intrusivo tramite Overlay e Dart Extensions.

4.3 Progettazione del database

Il modello concettuale del dominio, descritto nel Capitolo 2, è stato tradotto in un modello fisico implementato su un database **PostgreSQL**, gestito tramite la piattaforma Supabase. La progettazione si è concentrata su tre pilastri: integrità dei dati, sicurezza e performance.

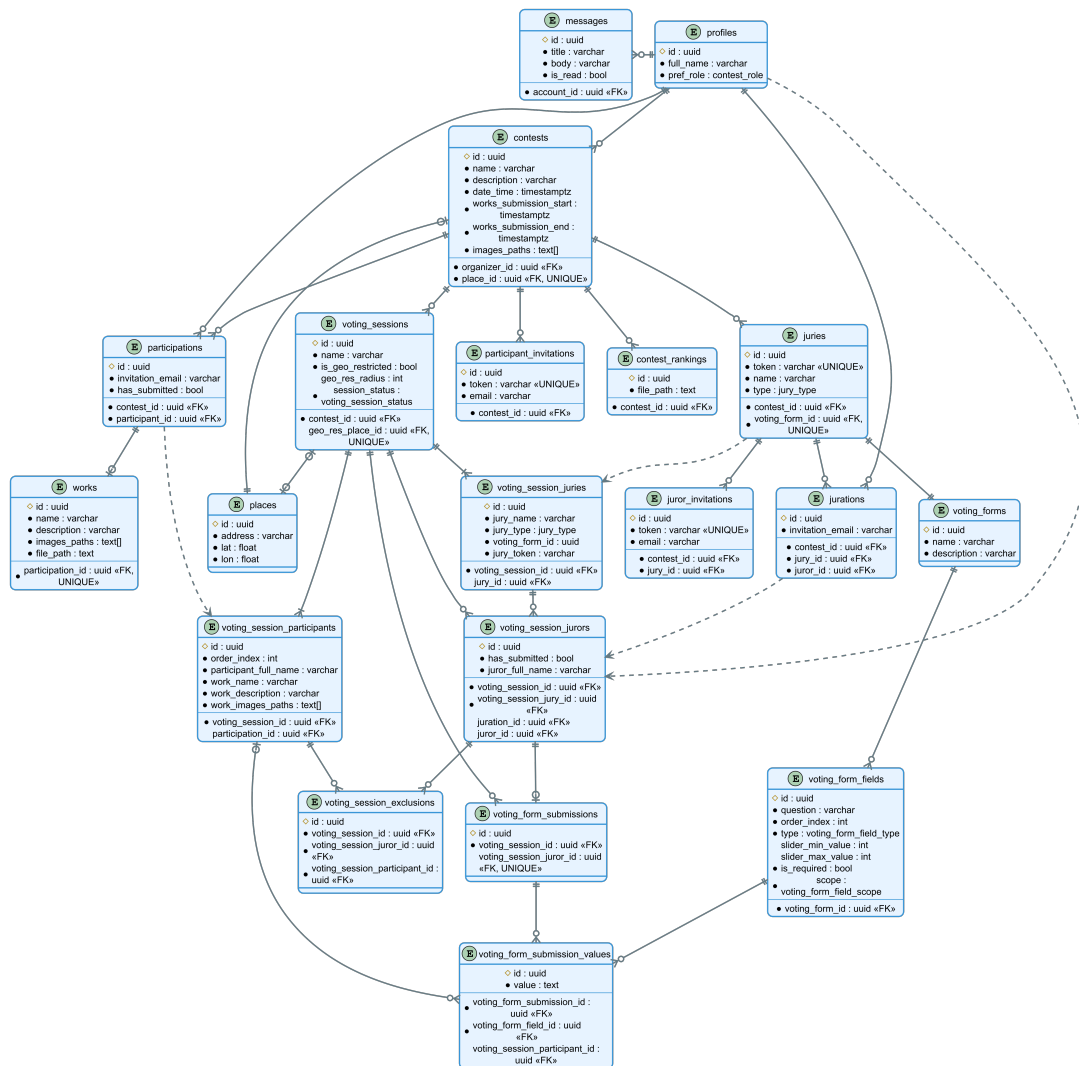


Figura 4.3: Diagramma Entità-Relazioni (ERD) dello schema "public" del database.

4.3.1 Schema fisico e tabelle principali

Lo schema del database è composto da una serie di tabelle che implementano le entità del dominio. Le tabelle cardine includono:

- **profiles:** Questa tabella estende la tabella `auth.users` gestita internamente da Supabase. Poiché lo schema `auth` è protetto e non modificabile, per aggiungere attributi personalizzati (es. `full_name`, `pref_role`) è stata creata una tabella parallela nello schema `public`. La colonna `id` di `profiles` funge sia da chiave primaria che da chiave esterna verso `auth.users(id)`, stabilendo una relazione uno-a-uno. La creazione del profilo è automatizzata tramite un **trigger di database**, che inserisce un nuovo record in `profiles` ogni volta che un utente completa la registrazione.
- **contests:** memorizza le informazioni principali di un concorso.
- **participations** e **jurations:** implementano le classi di associazione che legano, rispettivamente, un utente a un contest (*partecipazione*) e un utente a una giuria (*appartenenza a una giuria*).
- **works:** contiene i dati dei lavori sottomessi dai partecipanti.
- **voting_forms** e **voting_form_fields:** definiscono la struttura dei moduli di voto.
- **voting_sessions:** rappresenta una sessione di voto.
- **voting_session_juries**, **voting_session_participants**, **voting_session_jurors:** rappresentano i membri coinvolti in una specifica sessione di voto, oltre a contenere gli snapshot dei dati per quella sessione.
- **voting_form_submissions** e **voting_form_submission_values:** memorizzano i voti inviati dai giurati.

4.3.2 Scelte di design chiave

Oltre alla semplice mappatura delle entità, sono state prese decisioni di design cruciali per garantire la robustezza del sistema.

Integrità referenziale e strategia di cancellazione

Per garantire la consistenza del database, è stato fatto un uso estensivo dei vincoli di chiave esterna. Una decisione di design fondamentale è stata quella di adottare una strategia di **cancellazione fisica (hard delete)** piuttosto che una di **cancellazione logica (soft delete)**, principalmente per ottimizzare l'uso delle risorse (spazio su database e storage) e per semplificare la logica delle query di lettura.

Data questa scelta, la gestione della cancellazione dei dati correlati è diventata un aspetto critico, risolto con un approccio a più livelli.

Cancellazione a cascata tramite ON DELETE CASCADE Per le relazioni di composizione forte e dirette, dove l'entità "figlia" non ha dipendenze esterne come *file* nello Storage, è stata utilizzata la politica **ON DELETE CASCADE**. L'esempio più calzante è la relazione tra `voting_forms` e `voting_form_fields`. Poiché i campi di un form non hanno senso di esistere senza il form stesso, l'eliminazione di un record in `voting_forms` provoca la cancellazione automatica e a cascata di tutti i record correlati in `voting_form_fields`. Questo approccio delega interamente la logica di pulizia al database, garantendo massima efficienza e consistenza dei dati con il minimo sforzo implementativo.

Cancellazione tramite trigger per relazioni uno-a-uno In alcune relazioni uno-a-uno, la chiave esterna è stata posizionata nella tabella concettualmente "principale" per motivi di design. L'esempio più significativo è la relazione tra `contests` e `places`: è la tabella `contests` a contenere il riferimento `place_id` con un vincolo `UNIQUE`.

Questa configurazione, sebbene sia logicamente corretta, impedisce di usare la politica **ON DELETE CASCADE** per eliminare un `place` quando il `contest` associato viene cancellato. Per risolvere questo problema e automatizzare la pulizia dei dati, è stato implementato un **trigger di database**. Questo trigger si attiva *dopo* (`AFTER`) l'eliminazione di un record dalla tabella `contests` ed esegue esplicitamente la cancellazione del record corrispondente nella tabella `places`, utilizzando l'ID del luogo del contest appena eliminato. In questo modo, si garantisce l'integrità referenziale e

si evita di lasciare record `places` orfani, delegando anche in questo caso la logica di pulizia al database.

Cancellazione di dati esterni tramite Edge Functions La cancellazione a cascata del database non può gestire dati esterni, come i file memorizzati in Supabase Storage. Per questo motivo, le operazioni di cancellazione più complesse, come l'eliminazione di un utente o di un contest, sono gestite da **Edge Functions** dedicate (es. `user-delete-account`, `admin-delete-contest`). Queste funzioni agiscono da orchestratori: dopo aver eliminato i record dal database (che a loro volta attivano le cancellazioni a cascata e i trigger), invocano l'API di Supabase Storage per **eliminare tutti i file associati** (es. immagini del contest, file dei lavori), garantendo una pulizia completa e prevenendo la comparsa di file "orfani". Questa logica non poteva essere implementata in una funzione RPC, poiché queste non hanno accesso all'API dello Storage.

Snapshot dei dati tramite denormalizzazione controllata

Per garantire l'immutabilità e l'auditabilità del processo di voto, è stato implementato un **meccanismo di snapshot dei dati**, una delle scelte architetturali più importanti del backend.

Quando un organizzatore avvia una *VotingSession* tramite la funzione RPC `organizer_start_voting_session`, il sistema non si limita a collegare le entità esistenti, ma esegue una forma di **denormalizzazione controllata**, *duplicando intenzionalmente* alcuni dati per disaccoppiare il processo di voto dallo stato corrente del contest. Questo processo viene applicato a tutte le entità chiave della sessione: per ogni **partecipante, giuria e giurato** coinvolto, vengono create nuove righe in tabelle di sessione dedicate `voting_session_participants`, (`voting_session_juries` e `voting_session_jurors`).

Queste nuove righe contengono non solo le chiavi esterne ai record originali, ma anche una copia statica degli attributi più importanti al momento dell'avvio. Ad esempio, la tabella `voting_session_juries` memorizza il nome della giuria (`jury_name`), mentre `voting_session_participants` include il nome del par-

tecipante (`participant_full_name`), il nome del suo lavoro (`work_name`), e così via.

Una scelta di design cruciale è l’uso della clausola **ON DELETE SET NULL** sulle chiavi esterne. Questo significa che se l’entità originale dovesse essere eliminata dal sistema (es. un partecipante), il record dello snapshot *non* verrebbe cancellato, ma manterrebbe solo il riferimento a NULL. Ciò dimostra che la vera “fonte di verità” per la sessione di voto sono gli attributi di snapshot, garantendo che i voti espressi rimangano validi e auditabili anche se le entità originali cessano di esistere.

In `voting_session_jurors`, in particolare, `juror_id` è fondamentale per tracciare se un giurato ha già votato, ma può essere impostato a NULL in caso di cancellazione dell’account senza invalidare il voto già espresso.

Questa tecnica garantisce:

1. **Integrità della sessione:** se un utente cambia nome, il nome “congelato” nello snapshot rimane invariato.
2. **Auditabilità e resilienza:** i voti rimangono validi e associati ai dati di snapshot anche se i record originali vengono eliminati.
3. **Performance ottimizzate:** i dati di snapshot vengono letti direttamente, evitando query con join.

```

1  -- Definizione della tabella `voting_session_participants`
2
3  CREATE TABLE public.voting_session_participants (
4      id uuid PRIMARY KEY DEFAULT extensions.uuid_generate_v4(),
5      created_at timestamptz NOT NULL DEFAULT now(),
6      voting_session_id uuid NOT NULL,
7      participation_id uuid,
8      order_index int NOT NULL,
9
10     -- --- Attributi di Snapshot ---
11     participant_full_name varchar(70) NOT NULL,
12     work_name varchar(50) NOT NULL,
13     work_description varchar(1000) NOT NULL,
14     work_images_paths text[] NOT NULL,
15
16     UNIQUE (voting_session_id, participation_id),
17     UNIQUE (voting_session_id, order_index),

```

```

18
19 FOREIGN KEY (voting_session_id) REFERENCES public.voting_sessions
    ↳ (id) ON DELETE CASCADE,
20 -- ON DELETE SET NULL: se la partecipazione originale viene
    ↳ eliminata,
21 -- lo snapshot sopravvive, garantendo l'integrità dei voti.
22 FOREIGN KEY (participation_id) REFERENCES public.participations (id)
    ↳ ON DELETE SET NULL
23 );

```

Listato 4.6: Definizione della tabella “voting_session_participants” per lo snapshot dei dati dei partecipanti.

```

1 -- Definizione della tabella `voting_session_juries`
2
3 CREATE TABLE public.voting_session_juries (
4     id uuid PRIMARY KEY DEFAULT extensions.uuid_generate_v4(),
5     created_at timestamptz NOT NULL DEFAULT now(),
6     voting_session_id uuid NOT NULL,
7     jury_id uuid,
8
9     -- --- Attributi di Snapshot ---
10    jury_name varchar(50) NOT NULL,
11    jury_type jury_type NOT NULL,
12
13    UNIQUE (voting_session_id, jury_id),
14
15    FOREIGN KEY (voting_session_id) REFERENCES public.voting_sessions
        ↳ (id) ON DELETE CASCADE,
16    -- ON DELETE SET NULL: se la giuria originale viene eliminata,
17    -- lo snapshot sopravvive per mantenere l'integrità dei voti.
18    FOREIGN KEY (jury_id) REFERENCES public.juries (id) ON DELETE SET
        ↳ NULL
19 );

```

Listato 4.7: Definizione della tabella “voting_session_juries” per lo snapshot dei dati delle giurie.

```

1 -- Definizione della tabella `voting_session_jurors`
2
3 CREATE TABLE public.voting_session_jurors (
4     id uuid PRIMARY KEY DEFAULT extensions.uuid_generate_v4(),
5     created_at timestamptz NOT NULL DEFAULT now(),

```

```

6      voting_session_id uuid NOT NULL,
7      voting_session_jury_id uuid NOT NULL,
8
9      -- Riferimento alla "juration" se è un giurato nominato.
10     juration_id uuid,
11     -- Riferimento al profilo (per giurati semplici e nominati).
12     juror_id uuid,
13
14     has_submitted bool NOT NULL DEFAULT false,
15
16     -- --- Attributo di Snapshot ---
17     juror_full_name varchar(70) NOT NULL,
18
19     UNIQUE (voting_session_jury_id, juror_id),
20
21     FOREIGN KEY (voting_session_id) REFERENCES public.voting_sessions
22     → (id) ON DELETE CASCADE,
23     FOREIGN KEY (voting_session_jury_id) REFERENCES
24     → public.voting_session_juries (id) ON DELETE CASCADE,
25     -- ON DELETE SET NULL: se il giurato viene rimosso dalla giuria,
26     -- il suo record di sessione sopravvive per mantenere la coerenza.
27     FOREIGN KEY (juration_id) REFERENCES public.jurations (id) ON DELETE
28     → SET NULL,
29     -- ON DELETE SET NULL: se l'account del giurato viene eliminato,
30     -- il suo voto rimane registrato ma anonimizzato.
31     FOREIGN KEY (juror_id) REFERENCES public.profiles (id) ON DELETE SET
32     → NULL
33 );

```

Listato 4.8: Definizione della tabella “voting_session_jurors” per lo snapshot dei dati dei giurati.

4.3.3 Implicazioni del design del form di voto sull’aggregazione dei risultati

Una scelta di design fondamentale, visibile nello schema del database, è che **ogni giuria è associata a un unico e specifico form di voto**. Questa flessibilità permette all’organizzatore di creare criteri di valutazione diversi per giurie diverse (es. una giuria tecnica con campi slider complessi e una giuria popolare con un singolo campo di preferenza).

Tuttavia, questa decisione architetturale ha un'implicazione diretta e importante su come i risultati possono essere processati e visualizzati, influenzando sia il backend che il frontend: **il sistema non può aggregare automaticamente i punteggi tra giurie diverse**, poiché i loro form di voto (e quindi i loro criteri e punteggi) non sono direttamente comparabili.

Di conseguenza, tutte le operazioni di analisi dei risultati, sia a livello di logica di backend (RPC) che di interfaccia utente (frontend), sono state progettate per operare a livello della singola giuria:

- La **generazione della classifica** avviene per ogni singola giuria.
- L'**esportazione dei risultati** in formato Excel è granulare e produce un file per ciascuna giuria.

Se l'organizzatore desidera creare una classifica "globale" che combini i risultati di più giurie, deve farlo esternamente, utilizzando i dati esportati. Questa è una limitazione consapevole del progetto, un compromesso accettato per garantire la massima flessibilità nella configurazione dei criteri di valutazione.

Per mantenere questa flessibilità anche nella fase finale, il sistema non impone una classifica globale, ma permette all'organizzatore di **pubblicare qualsiasi classifica in formato PDF**. Questa scelta di design gli conferisce la **massima libertà**: può decidere di pubblicare la classifica di una singola giuria, oppure può elaborare esternamente una classifica globale personalizzata e caricarla come file PDF. In entrambi i casi, una volta pubblicata, la classifica diventa disponibile per il download a tutti i partecipanti e giurati del contest all'interno dell'applicazione.

4.4 Architettura del backend

Il backend è costruito interamente su Supabase e la sua architettura è definita da un'API sicura composta da un insieme di Funzioni RPC ed Edge Functions.

4.4.1 Funzioni RPC (Remote Procedure Call)

Le funzioni RPC, scritte in PL/pgSQL, sono il principale punto di ingresso per le operazioni sul database. La scelta di utilizzare le RPC come strato di accesso ai dati primario, invece di eseguire query dirette dal client, è stata dettata da tre motivi fondamentali:

1. **Sicurezza e Controllo:** Sono tutte dichiarate con `SECURITY DEFINER`, il che significa che vengono eseguite con i privilegi del loro creatore (un ruolo amministrativo) e non con quelli dell'utente chiamante. Al loro interno, contengono logica di validazione e controllo dei permessi, tipicamente verificando l'identità dell'utente tramite `auth.uid()`. Questa scelta è il fulcro dell'approccio di **accesso mediato da API**: nessun utente può effettuare operazioni dirette sul database, ma deve passare attraverso questo strato di controllo.
2. **Supporto per le Transazioni:** Una limitazione chiave della libreria client `supabase-flutter` è la **mancanza di supporto per le transazioni multi-statement**. Operazioni complesse che richiedono l'inserimento o l'aggiornamento di record su più tabelle in modo atomico (es. la creazione di un contest e del suo luogo associato) non sarebbero sicure se eseguite tramite chiamate multiple dal client. Implementando questa logica all'interno di una singola funzione RPC, si sfrutta la capacità nativa di PostgreSQL di eseguire tutte le operazioni in un'unica transazione, garantendo che l'operazione abbia successo o fallisca nella sua interezza, senza lasciare il database in uno stato inconsistente.
3. **Performance e Astrazione:** Le RPC permettono di incapsulare logica complessa e query con join multipli lato server, riducendo la quantità di dati trasferiti e il numero di round-trip tra client e server.

Nome Funzione	Scopo Principale	Controllo di Sicurezza
<code>auth_get_account_bundle</code>	Ottiene i dati completi dell'utente autenticato.	<code>auth.uid() = id</code>
<code>organizer_create_contest</code>	Crea un nuovo contest e il luogo associato.	Verifica che l'utente sia autenticato.
<code>organizer_start_voting_session</code>	Avvia una sessione di voto, creando lo snapshot dei dati.	L'utente deve essere l'organizzatore del contest.
<code>participant_join_contest</code>	Associa un partecipante a un contest tramite un token di invito.	Il token deve essere valido e non utilizzato.
<code>juror_submit_votes</code>	Salva i voti di un giurato per una sessione.	La sessione deve essere 'live' e il giurato non deve aver già votato.
<code>juror_access_voting_as_simple_juror</code>	Fornisce l'accesso a un giurato ospite tramite il token della giuria.	Il token della giuria deve essere valido.

Tabella 4.1: Elenco esemplificativo delle funzioni RPC del sistema.

A seguire, un esempio concreto di una funzione RPC, `organizer_cancel_voting_session`. Questo listato è particolarmente significativo perché dimostra in modo chiaro e conciso il pattern di sicurezza adottato: la funzione, eseguita come SECURITY DEFINER, verifica esplicitamente che l'utente chiamante sia l'organizzatore del contest prima di procedere con qualsiasi modifica, implementando così il principio di accesso mediato da API.

```

1 CREATE OR REPLACE FUNCTION organizer_cancel_voting_session
  ↪ (p_voting_session_id uuid)
2 RETURNS void
3 LANGUAGE plpgsql
4 SECURITY DEFINER SET search_path = public, extensions
5 AS $$
6 BEGIN
7     -- SECURITY CHECK: Verifica che l'utente sia l'organizzatore del
  ↪ contest.
8     IF NOT EXISTS (
9         SELECT 1
10        FROM public.voting_sessions vs
11        JOIN public.contests c ON vs.contest_id = c.id
12        WHERE vs.id = p_voting_session_id AND c.organizer_id = auth.uid()
13    ) THEN
14        RAISE EXCEPTION 'Voting session not found or access denied.';
15    END IF;
16
17    -- Se il controllo passa, procede con l'aggiornamento.
18    UPDATE public.voting_sessions
19    SET
20        session_status = 'cancelled'
21    WHERE id = p_voting_session_id;
22 END;
23 $$;

```

Listato 4.9: Implementazione della funzione RPC “organizer_cancel_voting_session”, che mostra il pattern di controllo dei permessi lato server.

4.4.2 Edge Functions

Le Edge Functions, scritte in TypeScript ed eseguite su Deno, gestiscono operazioni complesse, transazioni multi-step e integrazioni con servizi esterni che non sarebbero possibili o sicure da eseguire in una semplice RPC.

Nome Funzione	Scopo	Autorizzazione
organizer-invite-juror	Invia un'email di invito a un giurato.	JWT + Ownership del contest.

Nome Funzione	Scopo	Autorizzazione
participant-submit-work	Gestisce il caricamento del file .zip del lavoro e delle immagini di anteprima.	JWT + Appartenenza al contest.
user-delete-account	Elimina un utente e tutti i suoi dati e file associati.	JWT (invocata dall'utente stesso).
admin-delete-account	Versione amministrativa dell'eliminazione account.	X-API-Key (per Re-tool).
google-places-search-suggestions	Funge da proxy sicuro per l'API di Google Places.	JWT.
get-latest-apk	Proxy sicuro per il download dell'APK da un repository GitHub privato.	JWT.

Tabella 4.2: Elenco esemplificativo delle Edge Functions del sistema.

Per illustrare concretamente il ruolo delle Edge Functions nell'interagire con servizi esterni, il listato seguente mostra un estratto significativo della funzione `organizer-invite-juror`. Questo esempio è particolarmente rappresentativo perché dimostra un flusso di lavoro che non sarebbe stato possibile implementare in una semplice RPC: la funzione prima interagisce con il database per generare un token e creare un record di invito, e successivamente invoca un servizio di terze parti (l'API di Resend) per inviare l'email di invito.

```

1 // Import e inizializzazione del client di Resend.
2 // La chiave API viene letta in modo sicuro dai segreti di Supabase.
3 import { Resend } from 'npm:resend';
4 const resend = new Resend(Deno.env.get('RESEND_API_KEY'));
5
6 // ... (all'interno della funzione Deno.serve, dopo aver validato l'input
7 // e aver creato il record di invito nel database)

```

```

8
9 try {
10   // ...
11   // Passaggio 5: Inserisce il nuovo invito nel DB e ottiene il token
   ↪ generato.
12   const { data: newInvitation, error: insertError } = await
   ↪ supabaseAdmin
13     .from('juror_invitations')
14     .insert({ contest_id, jury_id, email })
15     .select()
16     .single();
17
18   if (insertError) throw insertError;
19
20   // Passaggio 6: Recupera i nomi del contest e della giuria per
   ↪ personalizzare l'email.
21   const { data: contest } = await
   ↪ supabaseAdmin.from('contests').select('name').eq('id',
   ↪ contest_id).single();
22   const { data: jury } = await
   ↪ supabaseAdmin.from('juries').select('name').eq('id',
   ↪ jury_id).single();
23
24   // Passaggio 7: Invia l'email di invito utilizzando il client di
   ↪ Resend.
25   // L'operazione è asincrona.
26   await resend.emails.send({
27     from: "Swift Contest <noreply@swiftcontest.com>",
28     to: [email],
29     subject: `Invitation to join the jury for "${contest.name}"`,
30     // Il corpo dell'email è un template HTML che include i dati
   ↪ dinamici.
31     html: `
32       <div>
33         <h1>Invitation to Swift Contest</h1>
34         <p>You have received an invitation to join the
   ↪ "<strong>${jury.name}</strong>" jury for the contest
   ↪ "<strong>${contest.name}</strong>".</p>
35         <p>Use this token in the app to accept:<br>
36         <strong>${newInvitation.token}</strong>
37       </p>
38     </div>
39     `,
40   });
41
42   // Passaggio 8: Se l'invio ha successo, restituisce l'invito creato
   ↪ al client.
43   return new Response(JSON.stringify(newInvitation), {

```

```
44     headers: { ...corsHeaders, "Content-Type": "application/json" },
45     status: 201, // Created
46   });
47
48 } catch (error) {
49   // Se l'invio dell'email o qualsiasi operazione precedente fallisce,
50   // l'errore viene catturato e registrato, e viene restituita una
51   // ↳ risposta di errore.
52   console.error("Error sending invitation:", error.message);
53   return new Response(JSON.stringify({ error: error.message }), {
54     headers: { ...corsHeaders, "Content-Type": "application/json" },
55     status: 500,
56   });
57 }
```

Listato 4.10: Estratto dalla Edge Function “organizer-invite-juror” che mostra l’invocazione dell’API di Resend.

4.5 Architettura di sicurezza: Principio del Minimo Privilegio

La sicurezza è un requisito trasversale che ha guidato l’intera progettazione del backend. È stato adottato un rigoroso modello basato sul **Principio del Minimo Privilegio (PoLP)**, il cui assunto fondamentale è negare l’accesso per default e concederlo solo in modo esplicito e controllato.

4.5.1 Strategia di implementazione: centralizzazione della logica di autorizzazione

Per implementare il Principio del Minimo Privilegio, si sarebbero potute seguire due strategie principali:

1. Definire policy di **Row Level Security (RLS)** [6] granulari, ovvero regole a livello di database che filtrano le righe visibili o modificabili da un utente in base alla sua identità o ai suoi permessi, per ogni tabella e per ogni operazione (SELECT, INSERT, UPDATE, DELETE).

2. Centralizzare tutti i controlli di autorizzazione all'interno di un'API di accesso ai dati (RPC e Edge Functions).

Sebbene un approccio ibrido con un doppio livello di controllo (sia RLS che controlli nell'API) rappresenti la massima sicurezza teorica, per questo progetto è stata adottata la **seconda strategia**. La logica di autorizzazione è stata interamente centralizzata all'interno delle Funzioni RPC e delle Edge Functions. Questa scelta è stata un **compromesso consapevole**, motivato dalla necessità di ottimizzare i tempi di sviluppo e la manutenibilità: gestire una logica di autorizzazione complessa in un unico punto (il codice della funzione) è risultato più rapido e meno prone a errori rispetto a dover mantenere sincronizzate decine di policy RLS scritte in SQL.

4.5.2 Il ruolo della RLS e dell'API come "Gatekeeper"

In questo modello, la RLS gioca comunque un ruolo fondamentale, ma più semplice: **agisce come un "firewall" di default**. Su tutte le tabelle è stata abilitata la RLS con una policy di `DENY` implicita. Questo garantisce che la chiave API usata dal client Flutter (`anon_key`) non abbia, di per sé, alcun permesso diretto di leggere o scrivere dati.

L'accesso ai dati è quindi interamente mediato da un'API sicura:

- **Funzioni RPC:** per la maggior parte delle operazioni sul database.
- **Edge Functions:** per interazioni con servizi esterni.

Queste funzioni, agendo da "gatekeeper", contengono al loro interno tutta la logica di autorizzazione (es. "l'utente è l'organizzatore di questo contest?"), verificando l'identità del chiamante tramite `auth.uid()` prima di eseguire qualsiasi operazione.

4.5.3 L'eccezione necessaria: le policy RLS per il servizio Realtime

Esiste un'eccezione mirata a questa strategia. Il servizio **Supabase Realtime** basa il suo meccanismo di autorizzazione proprio sulle policy RLS della tabella. Per permettere a un client di "ascoltare" le modifiche, è stato necessario definire policy di `SELECT` granulari, ma esclusivamente per le due tabelle esposte al servizio Realtime:

messages e voting_sessions. Questo è stato un adattamento necessario per integrare una funzionalità chiave, mantenendo la massima sicurezza su tutte le altre operazioni e tabelle.

Il seguente listato mostra la policy implementata per la tabella voting_sessions.

```

1  -- Abilita l'accesso in lettura alla tabella `voting_sessions`
2  -- solo per gli utenti autorizzati a ricevere aggiornamenti in tempo
   ↳ reale.
3  CREATE POLICY "Allow read access to organizers and involved jurors"
4  ON public.voting_sessions
5  FOR SELECT
6  USING (
7      -- Condizione 1: L'utente è l'organizzatore del contest a cui la
   ↳ sessione appartiene.
8      (EXISTS (
9          SELECT 1 FROM public.contests c
10         WHERE c.id = voting_sessions.contest_id AND c.organizer_id =
   ↳ auth.uid()
11     ))
12     OR
13     -- Condizione 2: L'utente è un giurato che partecipa a questa specifica
   ↳ sessione di voto.
14     (EXISTS (
15         SELECT 1 FROM public.voting_session_jurors vsj
16         WHERE vsj.voting_session_id = voting_sessions.id AND vsj.juror_id =
   ↳ auth.uid()
17     ))
18 );

```

Listato 4.11: Policy RLS di tipo SELECT per la tabella “voting_sessions”, necessaria per il servizio Realtime.

4.5.4 Misure di sicurezza aggiuntive

Oltre al modello ibrido RLS/RPC, la sicurezza del sistema è rafforzata da altre misure:

- **Protezione delle API Esterne:** le chiavi segrete per servizi come Resend e Google Places sono memorizzate in Supabase e accessibili solo dalle Edge Functions.

- **Protezione dei file:** nessun file nello Storage è pubblico. L'accesso avviene solo tramite *signed URL* (URL firmati), ovvero link **a tempo limitato e con permessi specifici**, generati dinamicamente lato server per ogni singola richiesta.

Gestione dei Giurati Semplici: il pattern dell'autenticazione anonima

Una delle sfide più complesse è stata quella di permettere a un *Giurato Semplice* di votare in modo sicuro senza obbligarlo a creare un account permanente. Risolvere questo problema solo con un token temporaneo sarebbe stato insicuro, poiché non avrebbe permesso di tracciare in modo affidabile se l'utente avesse già votato.

Per risolvere questa sfida, è stato implementato un pattern basato sull'**autenticazione anonima** offerta da Supabase. Questo approccio è un'applicazione diretta del Principio del Minimo Privilegio. Il flusso è il seguente:

1. **Creazione dell'account anonimo:** Nella pagina di accesso, l'utente seleziona l'opzione per partecipare come giurato semplice. L'applicazione gli chiede di inserire il proprio nome e cognome. Subito dopo, il client esegue una chiamata a Supabase per effettuare un **sign-in anonimo**.
2. **Autenticazione e Sessione JWT:** Supabase Auth crea un account utente a tutti gli effetti, ma marcato come "anonimo", e restituisce un JWT valido. Da questo momento, anche l'utente anonimo ha un `auth.uid()` univoco e una sessione autenticata. L'`AuthBloc` nel client entra in uno stato "autenticato ma anonimo".
3. **Accesso alla Home Page Limitata:** L'utente viene reindirizzato a una home page specifica per giurati semplici. L'`AuthGuard`, riconoscendo lo stato "anonimo", applica una logica di navigazione restrittiva: permette l'accesso solo a questa home page e alle pagine strettamente necessarie per la votazione, bloccando tutte le altre sezioni dell'app (es. impostazioni, gestione contest).
4. **Accesso alla Sessione di Voto:** Dalla sua home page, il giurato semplice ha due opzioni per accedere alla sessione di voto: può **scannerizzare un QR code** fornito dall'organizzatore o **inserire manualmente il token** della giuria.

5. **Validazione e Votazione:** Una volta inserito il token (tramite QR o manualmente), la RPC `juror_access_voting_as_simple_juror` lo valida. Se il token è corretto, l'utente viene reindirizzato alla procedura di voto, dove può esprimere le sue preferenze.
6. **Cancellazione dell'account al logout:** Al termine della procedura di voto, l'utente viene reindirizzato nuovamente alla sua home page limitata. Se a questo punto effettua il *logout*, viene invocata la Edge Function `user-delete-account`. Poiché l'utente possiede un JWT valido, è autorizzato a chiamare questa funzione, che elimina in modo sicuro e permanente l'account anonimo appena utilizzato, garantendo che nessun dato temporaneo rimanga nel sistema.

Questo pattern, oltre a garantire un'esperienza utente fluida e un alto livello di sicurezza, è stato progettato con un'importante considerazione sulla **scalabilità futura**. La gestione del giurato semplice tramite un vero account, seppur anonimo, pone le basi per future evoluzioni, come l'implementazione di una funzionalità che permetta all'utente di "reclamare" il proprio account anonimo e **convertirlo in un account permanente**.

I vantaggi principali di questo approccio sono:

- **Esperienza Utente Fluida:** L'utente non percepisce la creazione di un account, ma solo l'inserimento del proprio nome, seguito da un'interfaccia chiara per accedere alla votazione.
- **Sicurezza Robusta:** Ogni azione del giurato semplice è autenticata e legata a un `uid` univoco, prevenendo abusi.
- **Riutilizzo e Scalabilità dell'architettura:** Permette di riutilizzare la stessa logica di sicurezza (RLS, RPC, `AuthGuard`) sia per gli utenti normali che per quelli anonimi e apre la strada a future evoluzioni del sistema di account.

4.6 Progettazione e implementazione degli strumenti di amministrazione

Oltre all'applicazione destinata agli utenti finali, un sistema software robusto richiede strumenti adeguati per la sua gestione, moderazione e manutenzione. In linea con l'obiettivo di concentrare le risorse di sviluppo sull'applicazione principale destinata agli utenti finali, per la gestione del sistema si è adottata una **strategia di sviluppo rapido**: invece di investire tempo nella creazione di un pannello di amministrazione da zero, si è scelto di utilizzare **Retool**, una piattaforma low-code specializzata nella creazione rapida di strumenti interni.

4.6.1 Esigenza di un pannello di amministrazione

Sebbene la dashboard di Supabase offra un accesso completo al database, essa è uno strumento pensato per gli sviluppatori, non per gli amministratori di sistema. La necessità di un pannello di amministrazione dedicato nasce quindi da requisiti operativi critici che, pur essendo tecnicamente possibili, **non sono gestibili in modo semplice, sicuro e rapido** tramite l'interfaccia standard di Supabase:

- **Gestione Utenti**: la capacità di visualizzare tutti gli utenti registrati, verificarne l'attività e, in caso di abuso, procedere con l'eliminazione dell'account.
- **Moderazione Contenuti**: la possibilità per un amministratore di ispezionare e, se necessario, rimuovere forzatamente un contest che viola i termini di servizio.

4.6.2 Scelta di Retool per lo sviluppo rapido

La scelta di Retool è stata strategica e motivata da:

- **Velocità di Sviluppo**: ha permesso di costruire un'interfaccia admin funzionale in poche ore, invece che in settimane, concentrando le risorse di sviluppo sull'applicazione principale.
- **Integrazione Nativa**: si integra nativamente con database PostgreSQL e API REST, permettendo di interfacciarsi facilmente con l'infrastruttura Supabase.

- **Sicurezza Configurabile:** offre meccanismi per gestire segreti (come le chiavi API) e configurare le chiamate in modo sicuro.

4.6.3 Architettura di comunicazione sicura tra Retool e Supabase

La sicurezza è stata la priorità nella progettazione dell'integrazione. L'accesso ai dati non è mai diretto, ma mediato da un'API sicura esposta dal backend Supabase, con due meccanismi distinti:

Accesso tramite utente dedicato e funzioni RPC

Per le operazioni che richiedono solo l'accesso al database (es. visualizzare la lista di utenti), è stato creato un utente PostgreSQL dedicato, `retool`, con permessi minimi. Le query da Retool invocano funzioni RPC con prefisso `admin_*`, che al loro interno verificano che il chiamante (`session_user`) sia esattamente `retool`, bloccando qualsiasi altro tentativo di accesso.

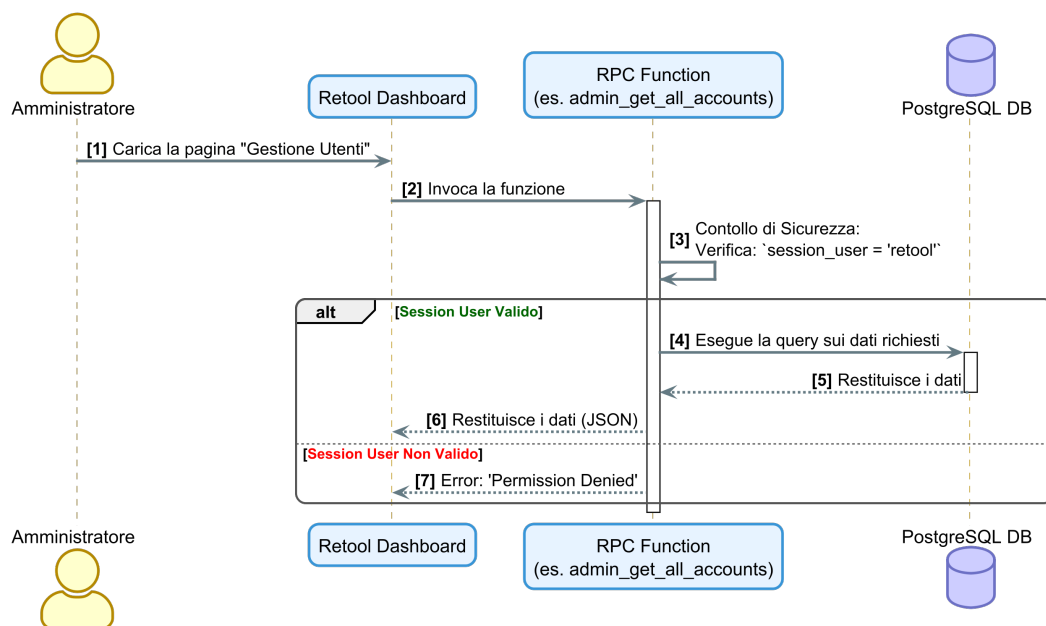


Figura 4.4: Flusso di una chiamata sicura da Retool a una Funzione RPC.

```

1 -- Estratto dalla funzione 'admin_get_all_accounts()'
2
3 CREATE OR REPLACE FUNCTION admin_get_all_accounts()
4 RETURNS SETOF profiles -- o un altro tipo di ritorno
  
```

```

5 LANGUAGE plpgsql
6 SECURITY DEFINER
7 SET search_path = public
8 AS $$
9 BEGIN
10     -- ---- 1. CONTROLLO DI SICUREZZA ----
11     -- Questa è la prima operazione eseguita dalla funzione.
12     -- 'session_user' è una variabile speciale di PostgreSQL che contiene
13     -- il nome dell'utente della connessione corrente (in questo caso,
14     -- ↪ 'retool').
15     IF session_user <> 'retool' THEN
16         -- Se il chiamante non è l'utente 'retool' designato,
17         -- la funzione solleva un'eccezione e interrompe immediatamente
18         -- ↪ l'esecuzione.
19         RAISE EXCEPTION 'Permission denied: This function can only be called
20         -- ↪ by the admin user.';
21     END IF;
22
23     -- ---- 2. ESECUZIONE DELLA LOGICA ----
24     -- Se il controllo di sicurezza passa, la funzione procede
25     -- con l'esecuzione della query per recuperare i dati.
26     RETURN QUERY
27     SELECT * FROM public.profiles;
28 $$;

```

Listato 4.12: Validazione del chiamante all'interno di una funzione RPC per l'amministrazione.

Accesso tramite Edge Functions e API Key

Per le operazioni che richiedono un'orchestrazione complessa o l'interazione con servizi esterni al database, come l'invio di email o la manipolazione di file nello Storage, sono state implementate delle **Edge Functions**. Funzioni critiche come `admin-delete-account` e `admin-delete-contest` rientrano in questa categoria, poiché devono non solo modificare i record del database, ma anche eliminare tutti i file associati (immagini, file) dallo Storage per garantire una pulizia completa.

Queste funzioni sono protette da un meccanismo di autenticazione server-to-server per garantire che possano essere invocate solo da un client autorizzato come

Retool:

1. Retool, nell'eseguire l'azione, invia una richiesta HTTP POST alla Edge Function, includendo un header personalizzato: X-API-Key.
2. Il valore di questa chiave è un segreto memorizzato sia in Retool che in Supabase.
3. La Edge Function, come primo passo, confronta la chiave ricevuta con quella memorizzata nei suoi segreti. Se non corrispondono, la richiesta viene immediatamente respinta con un errore 401 (Unauthorized).

Questo approccio garantisce che solo un'applicazione autorizzata possa invocare queste funzioni, proteggendo il sistema da accessi impropri.

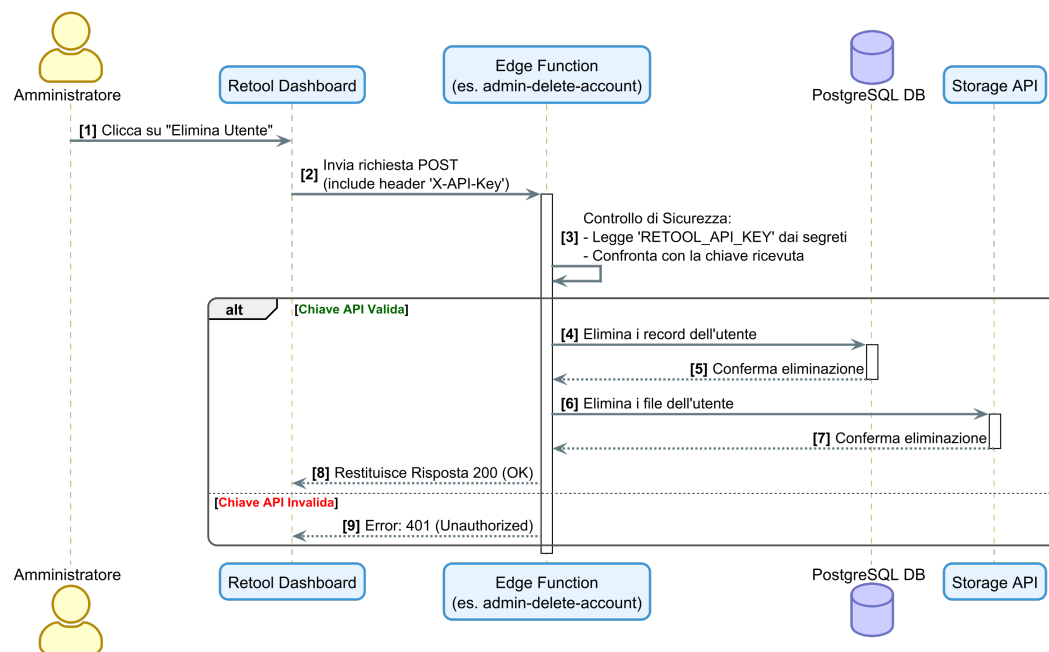


Figura 4.5: Flusso di una chiamata sicura da Retool a una Edge Function, protetta da una chiave API.

```

1 // Estratto dalla Edge Function 'admin-delete-account/index.ts'
2
3 // ... (all'interno della funzione Deno.serve)
4 try {
5   // --- 1. API KEY AUTHORIZATION ---
  
```

```
6 // Questo pattern è usato per tutte le chiamate server-to-server da
   ↳ Retool.
7
8 // Legge la chiave API inviata dal client (Retool) nell'header.
9 const apiKey = req.headers.get('X-API-Key');
10
11 // Recupera la chiave segreta memorizzata in modo sicuro nel Vault di
   ↳ Supabase.
12 const secretKey = Deno.env.get('RETOOL_API_KEY');
13
14 // Primo controllo: verifica che il segreto sia stato configurato.
15 if (!secretKey) {
16   throw new Error("RETOOL_API_KEY is not set in Supabase secrets.");
17 }
18
19 // Secondo controllo: confronta la chiave ricevuta con il segreto.
20 // Se non corrispondono, la richiesta viene immediatamente respinta.
21 if (apiKey !== secretKey) {
22   return new Response(
23     JSON.stringify({ error: "Unauthorized. Invalid API Key." }),
24     {
25       headers: { ...corsHeaders, "Content-Type": "application/json" },
26       status: 401, // 401 Unauthorized
27     }
28   );
29 }
30
31 // --- 2. PROSECUZIONE DELLA LOGICA ---
32 // Se la validazione ha successo, la funzione procede con
33 // l'esecuzione delle operazioni amministrative.
34
35 // ... (logica per eliminare l'utente, i file, ecc.)
36
37 } catch (error) {
38   // ... (gestione degli errori)
39 }
```

Listato 4.13: Validazione della chiave API all'interno di una Edge Function amministrativa.

4.6.4 Funzionalità implementate nel pannello

La dashboard sviluppata in Retool fornisce all'amministratore le seguenti funzionalità chiave:

- **Una tabella** per la visualizzazione e la ricerca di tutti gli utenti registrati.

- **Una vista di dettaglio** per un singolo utente, che mostra i contest a cui è associato e i suoi ruoli.
- **Un pulsante “Elimina Utente”**, che invoca in modo sicuro la Edge Function `admin-delete-account`.
- **Una tabella di tutti i contest**, con un pulsante “Elimina Contest” che invoca la Edge Function `admin-delete-contest`.

Distribuzione e prodotto finale

5.1 Strategia di distribuzione

La fase di distribuzione ha avuto come obiettivo non solo il rilascio tecnico dell'applicazione, ma anche la validazione della sua utilizzabilità in un ambiente che simula condizioni reali di produzione. Per **Swift Contest** è stato scelto un approccio multiplatforma che copre i principali punti di accesso per i diversi tipi di utenti:

- **Applicazione Web:** distribuita su **Vercel**, per garantire accessibilità universale da qualsiasi browser moderno, sia desktop che mobile.
- **Applicazione Android:** distribuita come pacchetto APK, reso disponibile tramite un meccanismo di download sicuro per un'esperienza d'uso ottimizzata su dispositivi mobili.
- **Infrastruttura Backend:** interamente gestita su **Supabase**, che ospita il database, lo storage, le funzioni serverless e i servizi realtime.
- **Pannello di Amministrazione:** sviluppato come strumento interno su **Retool** e interfacciato in modo sicuro con il backend, per le attività di moderazione e manutenzione.

Questa strategia ibrida consente di bilanciare la facilità di accesso (garantita dalla versione Web) con un'esperienza d'uso ottimizzata (offerta dalla versione Android), mantenendo al contempo un backend centralizzato, sicuro e scalabile.

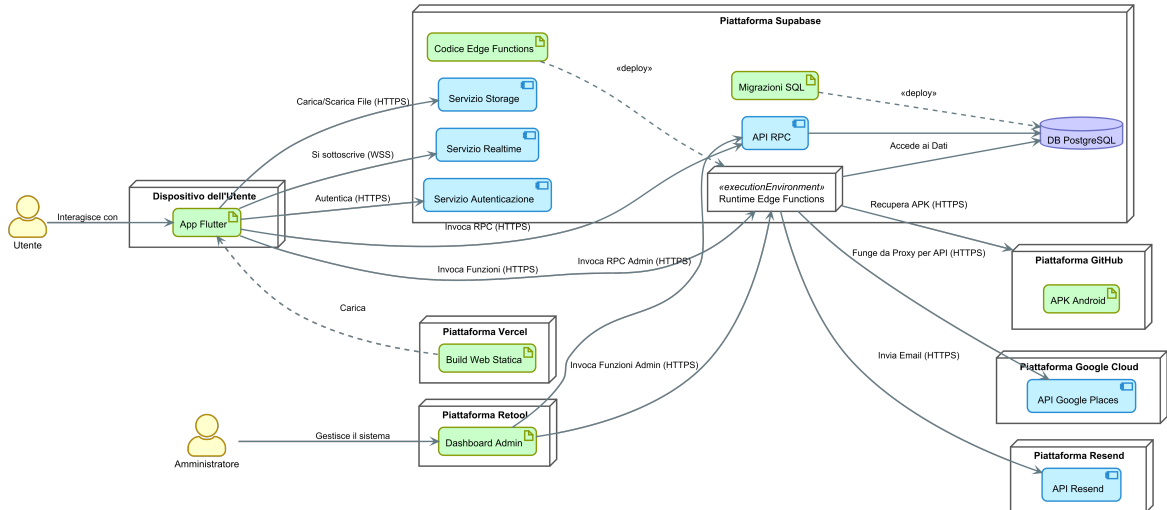


Figura 5.1: Diagramma di distribuzione dell'architettura completa di Swift Contest.

5.2 Distribuzione web su Vercel

La versione Web dell'applicazione Flutter è stata distribuita su Vercel [9], una piattaforma serverless ottimizzata per la distribuzione di frontend moderni. Il processo è completamente automatizzato tramite una pipeline di **CI/CD (Continuous Integration/Continuous Deployment)** integrata con il repository Git del progetto. Ogni *push* sul branch `main` innesca automaticamente una nuova build dell'applicazione Flutter in modalità *release* e una conseguente distribuzione sulla piattaforma.

Per garantire il corretto funzionamento di una *Single Page Application (SPA)* come Flutter Web, è stato configurato un file `vercel.json` che gestisce il rewrite di tutte le rotte verso il file `index.html`, permettendo al router di Flutter (`auto_route`) di gestire la navigazione lato client.

5.3 Distribuzione Android

Per la piattaforma Android, è stata prodotta una **release in formato APK**. Data la natura prototipale del progetto, si è evitato il complesso processo di pubblicazione

sul Google Play Store, optando per un approccio di distribuzione controllata che garantisce comunque sicurezza e facilità di aggiornamento:

1. L'APK viene caricato come *release asset* (un file allegato a una specifica versione del software) su un repository GitHub [16] privato.
2. Un'apposita Edge Function, `get-latest-apk`, è stata creata su Supabase per fungere da proxy sicuro. Questa funzione utilizza un **Personal Access Token (PAT)** di GitHub, memorizzato in modo sicuro in Supabase, per autenticarsi all'API di GitHub e scaricare l'asset.
3. L'applicazione web offre un pulsante "Download for Android" che invoca questa Edge Function. L'utente riceve così l'ultima versione dell'APK direttamente dal backend, senza che il repository GitHub o le credenziali di accesso vengano mai esposte.

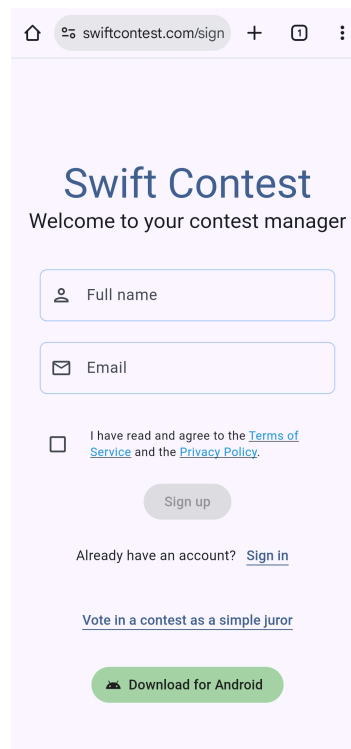


Figura 5.2: Pagina di accesso dell'applicazione web distribuita su Vercel.

5.4 Backend e Supabase

L'intera infrastruttura backend è gestita su Supabase. Il processo di distribuzione del database è basato su migrazioni SQL, seguendo le pratiche di **Infrastructure as Code (IaC)**. Ogni modifica allo schema del database, alle policy RLS o alle funzioni RPC è definita in un file `.sql` all'interno della cartella `supabase/migrations`. La CLI di Supabase viene utilizzata per applicare queste migrazioni in modo sequenziale e versionato, garantendo che l'ambiente di produzione sia sempre allineato con la definizione del codice. Le Edge Functions, scritte in TypeScript, vengono anch'esse distribuite tramite la CLI.

Componente	Utilizzo nel Progetto Swift Con-test	Implementazione / Note Tecniche
Database (PostgreSQL)	Persistenza di tutti i dati applicativi: utenti, contest, partecipazioni, giurie, voti, ecc. È il "single source of truth" del sistema.	Schema e logica definiti tramite file di migrazione SQL nella cartella <code>supabase/migrations</code> .
Auth	Gestione completa del ciclo di vita degli utenti: registrazione, login, autenticazione anonima e gestione delle sessioni tramite JWT.	Integrato tramite la libreria <code>supabase-flutter</code> . La tabella <code>auth.users</code> è estesa dalla tabella <code>public.profiles</code> .
Storage	Archiviazione sicura di tutti i file caricati dagli utenti, come le immagini dei contest e i file dei lavori.	Nessun file è pubblico. L'accesso avviene esclusivamente tramite <i>signed URL</i> temporanei generati lato server.

Componente	Utilizzo nel Progetto Swift Con-test	Implementazione / Note Tecniche
Edge Functions	Esecuzione di logica server-side complessa, orchestrazione di operazioni multi-servizio e interazione con API esterne.	Scritte in TypeScript (runtime Deno) e distribuite dalla cartella <code>supabase/functions</code> .
Funzioni RPC	Punto di ingresso primario per la logica di business confinata al database.	Scritte in PL/pgSQL e definite nei file di migrazione.
Realtime	Propagazione di modifiche del database ai client sottoscritti in tempo reale per notifiche e aggiornamenti di stato.	L'autorizzazione alla sottoscrizione è gestita tramite policy RLS di tipo <code>SELECT</code> .

Tabella 5.1: Riepilogo delle componenti Supabase e del loro utilizzo nel progetto.

5.5 Dashboard amministrativa (Retool)

Per le attività di amministrazione avanzata, come la moderazione degli utenti e la gestione dei contenuti, è stata creata una dashboard interna utilizzando **Retool** [14]. Questa interfaccia, non esposta agli utenti finali, comunica con il backend Supabase attraverso un'API sicura, come descritto nel capitolo precedente.

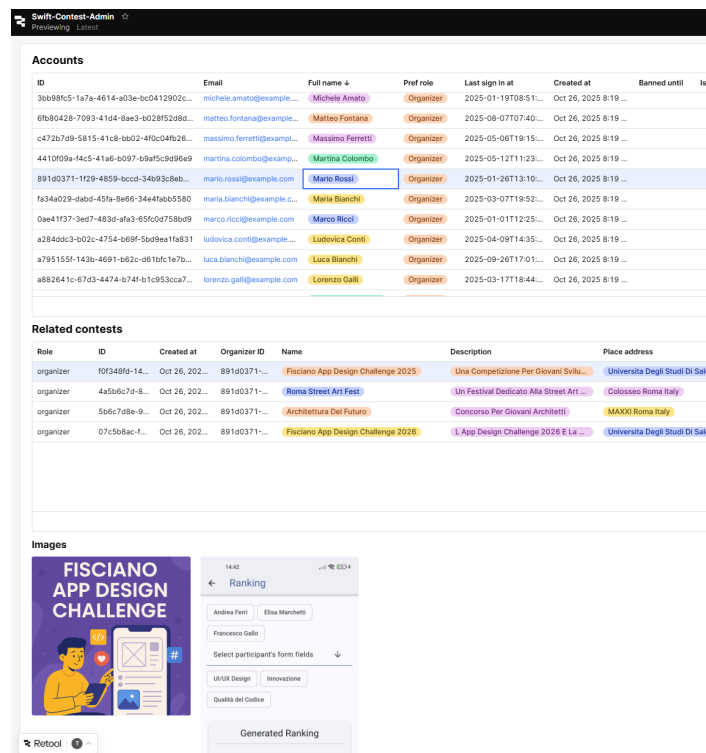


Figura 5.3: La dashboard di amministrazione sviluppata in Retool, che mostra l'elenco degli utenti registrati.

5.6 Prodotto finale: interfaccia utente e flussi operativi

In questa sezione viene presentato il prodotto finale attraverso un'analisi visuale dei suoi **flussi operativi principali**. L'obiettivo non è quello di documentare ogni singola schermata dell'applicazione, ma di illustrare, tramite l'interfaccia utente (UI), i percorsi più significativi per ciascuno degli attori del sistema.

Partendo dalle schermate di base, come l'autenticazione e le impostazioni, verranno poi illustrati i flussi specifici per l'**Organizzatore**, il **Partecipante**, il **Giurato Nominato** e il **Giurato Semplice** (ospite).

5.6.1 Accesso e autenticazione

L'accesso all'applicazione inizia con una **schermata di avvio (Splash Screen)**. Questa schermata, oltre a gestire l'inizializzazione dei servizi principali, presenta l'identità visiva del progetto attraverso il suo **logo**. Il design del logo si basa su una coppa stilizzata, simbolo universale di competizione, i cui manici sono modellati per formare le lettere "S" e "C", le iniziali di Swift Contest.

Successivamente, l'utente viene indirizzato al flusso di autenticazione, che comprende le pagine di **Accesso e Registrazione**.

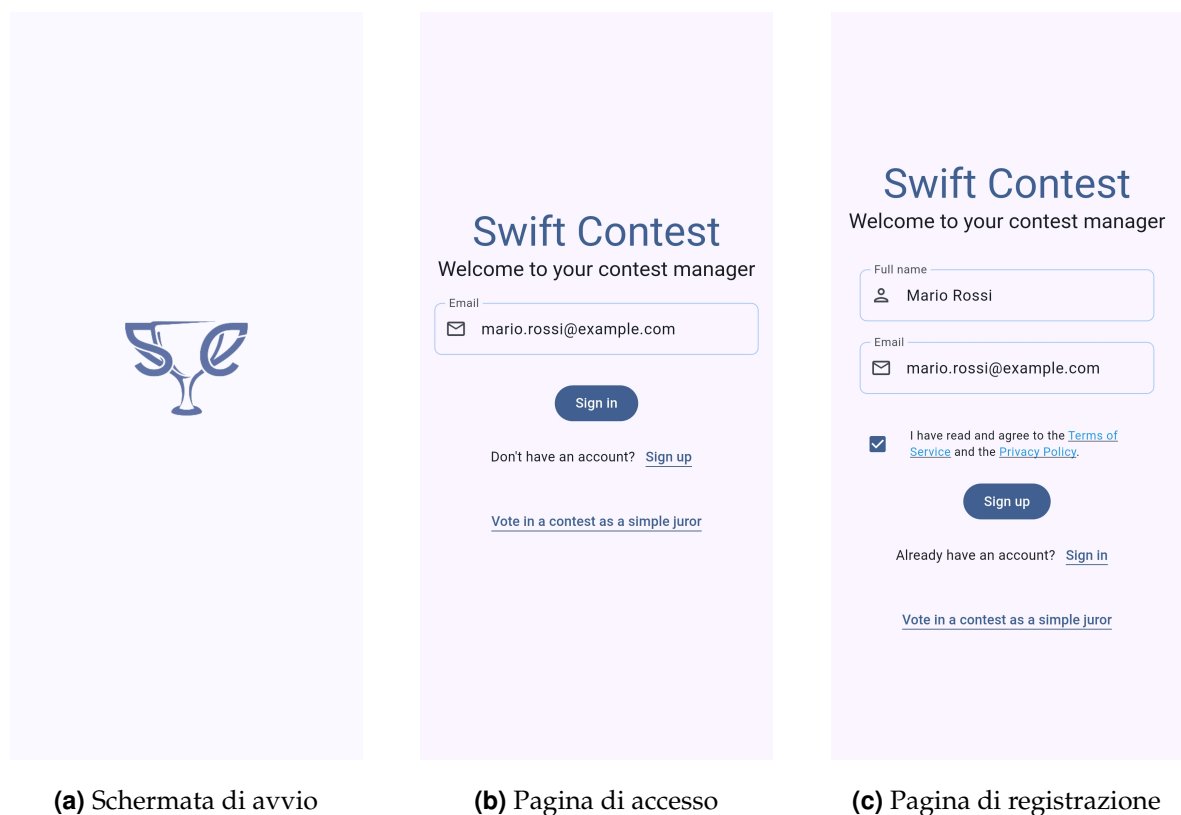


Figura 5.4: Le schermate del flusso di autenticazione dell'applicazione.

5.6.2 Gestione delle impostazioni

Una volta autenticato, ogni utente ha accesso a una sezione dedicata alle impostazioni. La pagina delle **Impostazioni Generali** offre diverse funzionalità:

- Permette di cambiare il **tema** dell'applicazione (chiaro, scuro o di sistema).

- Consente di impostare il **ruolo preferito** (es. Organizzatore, Partecipante), che determina quale dashboard verrà mostrata all'avvio dell'app.
- Fornisce l'accesso ai documenti legali, come i **Termini di Servizio** e l'**Informativa sulla Privacy**.
- Contiene il pulsante per effettuare il **logout**.

Da qui, l'utente può inoltre navigare alla pagina **Impostazioni Account**, dove può modificare i dati del proprio profilo o procedere con l'eliminazione sicura del proprio account.

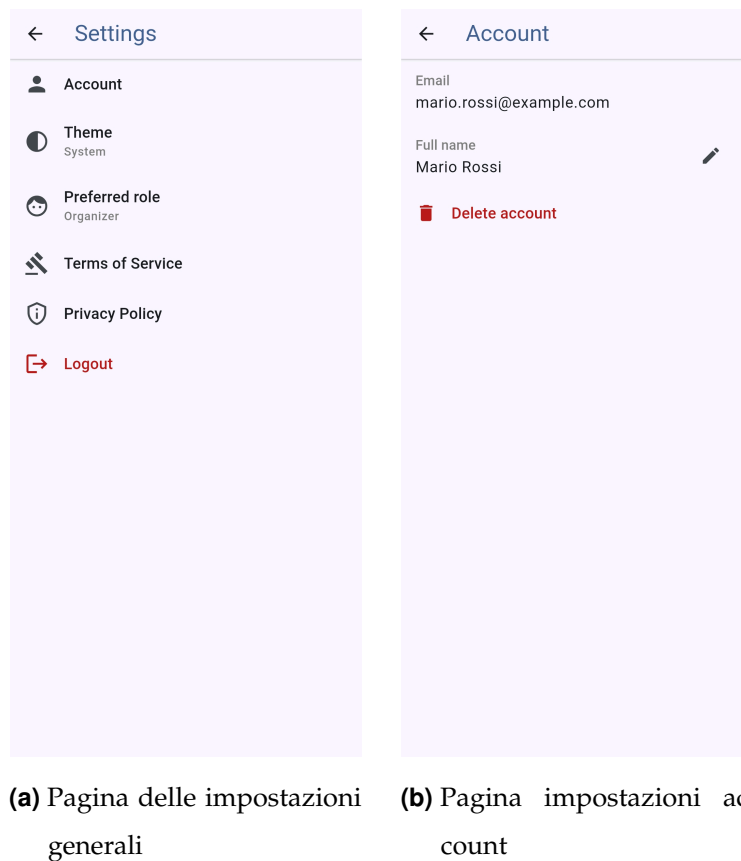


Figura 5.5: Le schermate per la gestione delle impostazioni dell'app e dell'account utente.

5.6.3 Flusso dell'organizzatore

L'organizzatore è l'attore con il set di funzionalità più ricco. Il suo percorso inizia dalla **Home Page**, che funge da dashboard principale, mostrando l'elenco dei contest da lui creati.

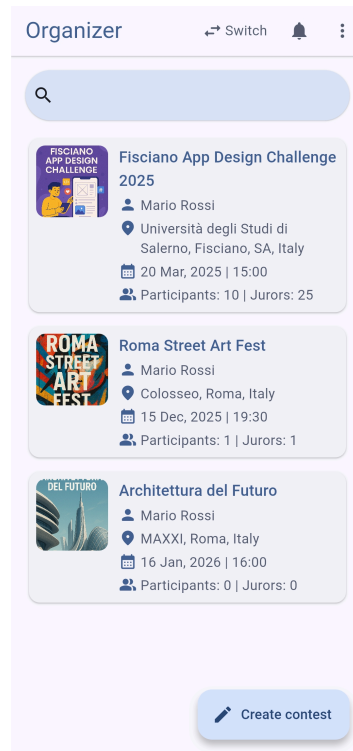


Figura 5.6: La home page dell'organizzatore.

Creazione di un nuovo contest Dalla home page, l'organizzatore può avviare la creazione di un nuovo contest tramite un **form multi-step** che semplifica l'inserimento delle informazioni, suddivise in tre passaggi logici:

1. **Dati Principali:** nome, descrizione, data e luogo del contest.
2. **Immagini:** caricamento delle immagini di copertina.
3. **Periodo di Sottomissione:** definizione delle date di inizio e fine per la sottomissione dei lavori.

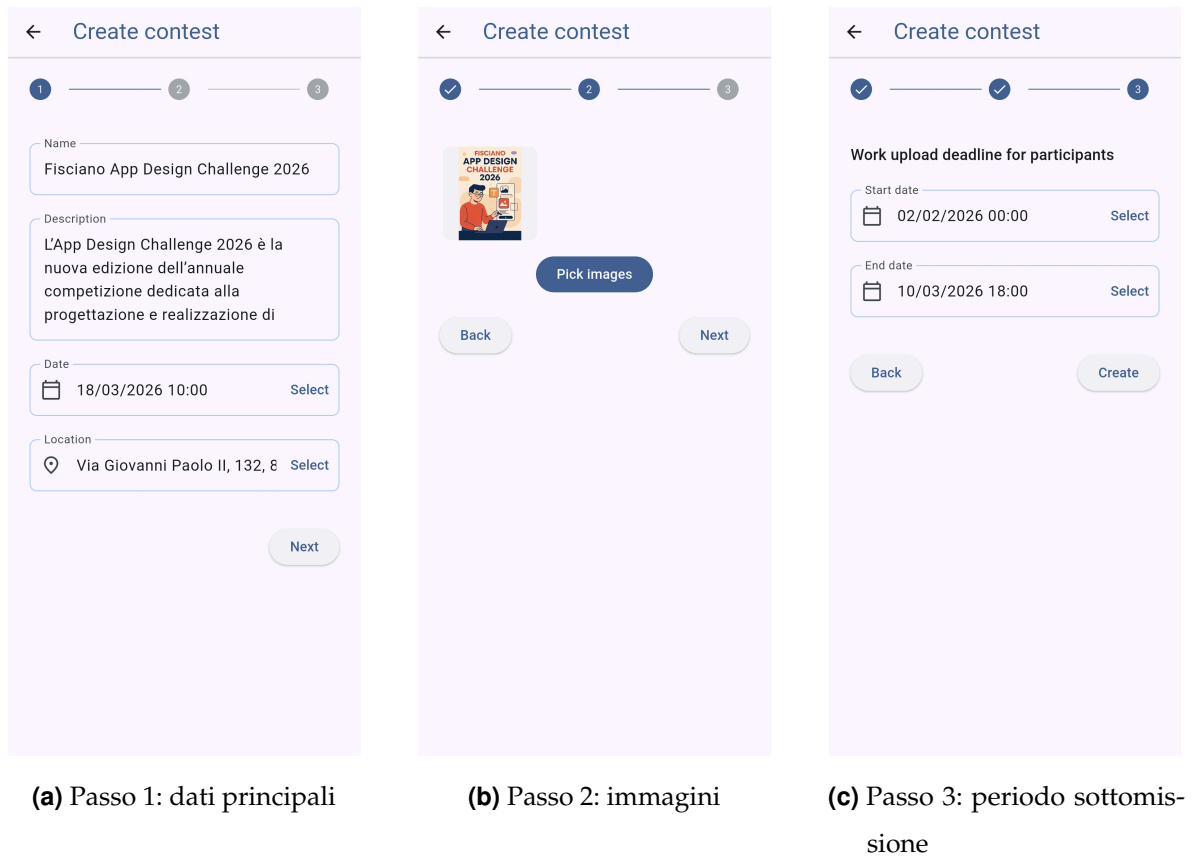
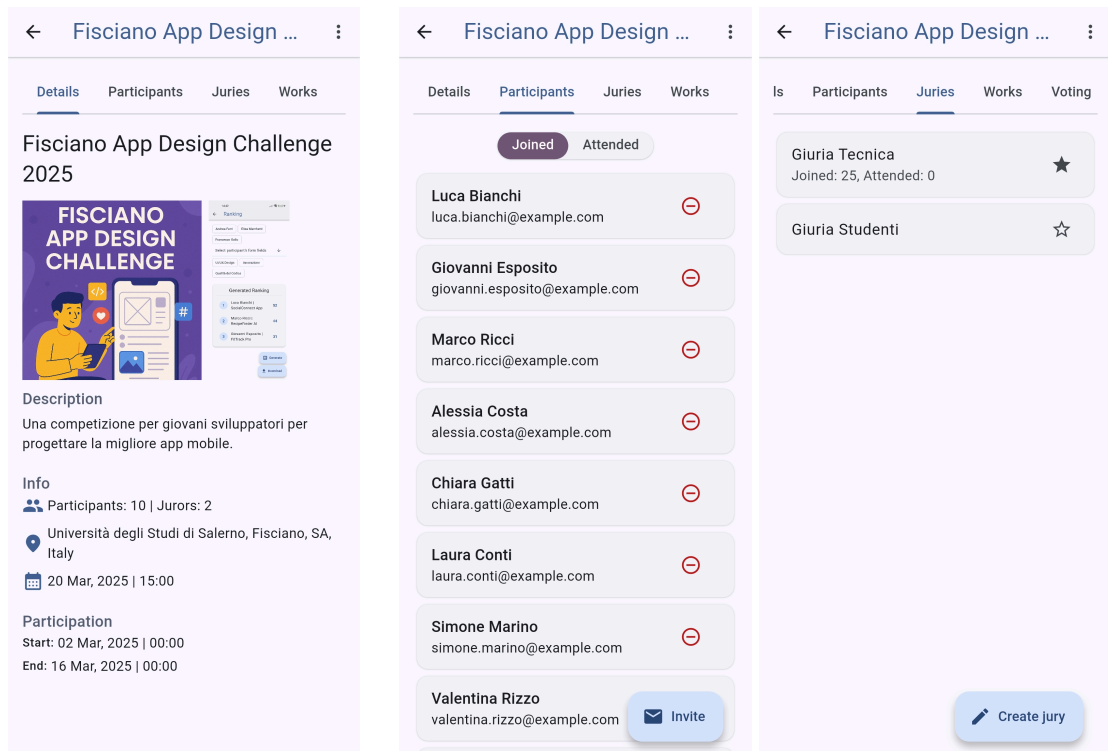


Figura 5.7: Le tre schermate del form guidato per la creazione di un nuovo contest.

Configurazione dei dettagli del contest Selezionando un contest, l'organizzatore accede al pannello di gestione, organizzato in tab, da cui può configurare ogni aspetto:

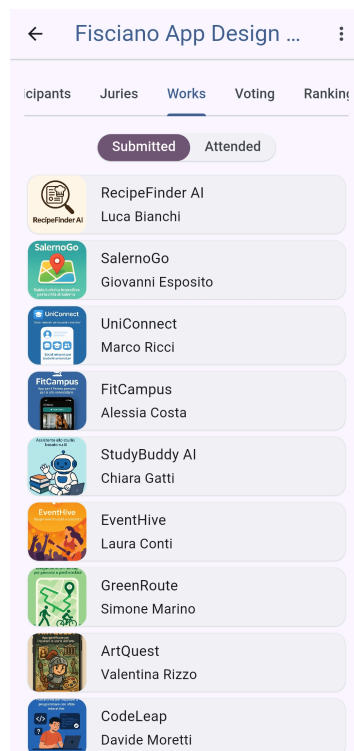
- La tab **Details** mostra un riepilogo delle informazioni generali.
- La tab **Participants** permette di gestire e invitare i partecipanti.
- La tab **Juries** consente di creare e gestire le giurie.
- La tab **Works** mostra l'elenco dei lavori sottomessi.



(a) Tab "Details"

(b) Tab "Participants"

(c) Tab "Juries"



(d) Tab "Works"

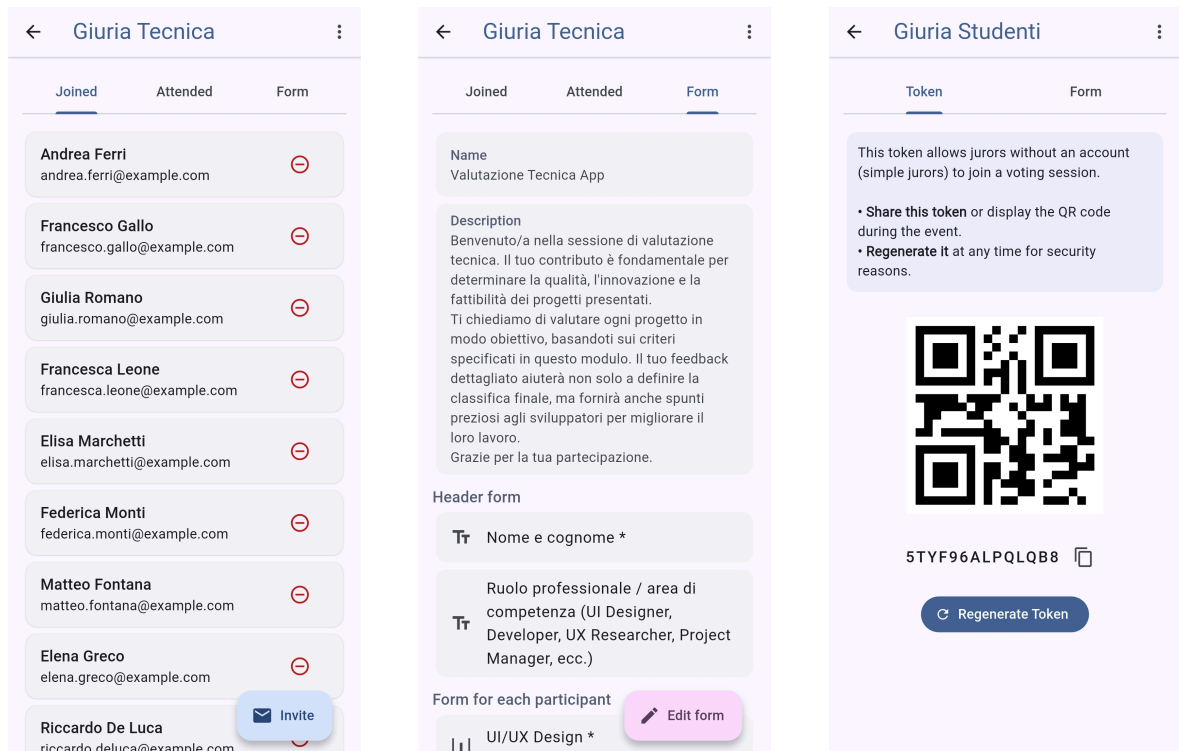
Figura 5.8: Le principali tab di gestione all'interno di un contest.

In particolare, la gestione delle giurie prevede una schermata di dettaglio il cui

contenuto varia a seconda del tipo di giuria:

- Per una **giuria nominata**, la pagina mostra l'elenco dei giurati.
- Per una **giuria semplice**, la pagina espone il **token** di accesso.

Per entrambe, è possibile accedere alla configurazione del **form di voto** associato.



(a) Dettagli giuria nominata

(b) Configurazione form di voto

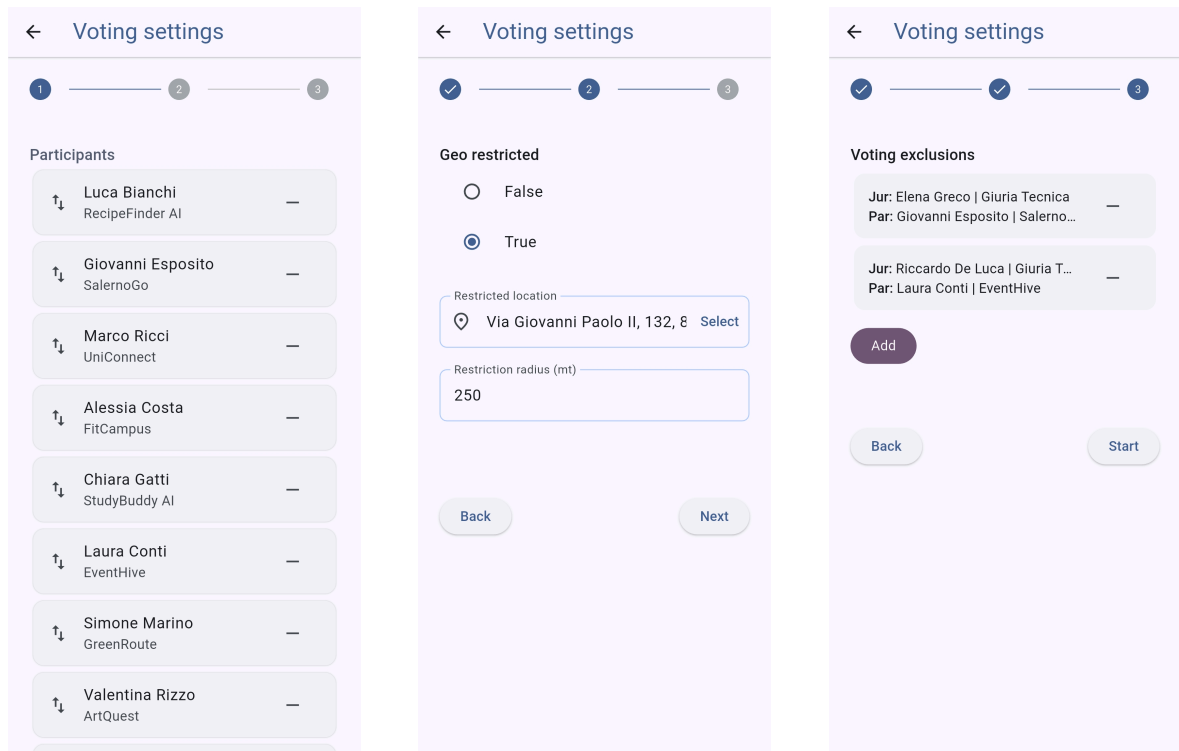
(c) Dettagli giuria semplice

Figura 5.9: Le viste di dettaglio per i due tipi di giuria e la configurazione del form di voto.

Gestione della sessione di voto La tab **Voting** permette di avviare una nuova sessione tramite un form a tre passaggi:

1. **Selezione dei partecipanti.**
2. **Restrizione geografica (opzionale).**
3. **Gestione esclusioni per i giurati nominati.**

Una volta avviata, la schermata mostra lo stato **Live** della sessione e il token per i giurati semplici.



(a) Passo 1: partecipanti

(b) Passo 2: restrizione geografica

(c) Passo 3: esclusioni

Figura 5.10: I passaggi per la configurazione di una nuova sessione di voto.

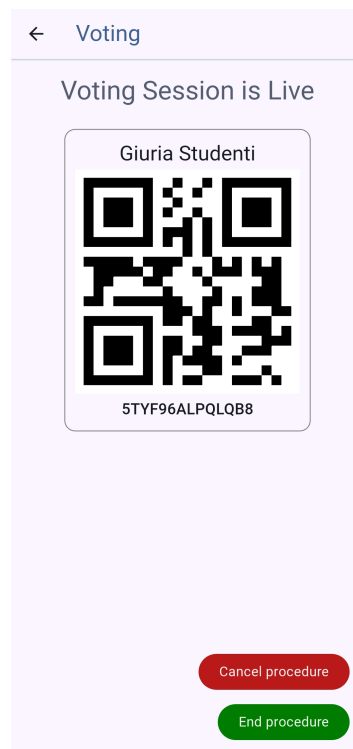


Figura 5.11: La schermata di una sessione di voto attiva (live).

Analisi dei risultati e classifiche Una volta che una sessione di voto è terminata, dalla tab **Voting** l'organizzatore può:

- **Visualizzare i risultati aggregati** per ciascuna giuria che ha partecipato alla sessione.
- **Esplorare i dettagli** di ogni sottomissione, visualizzando i voti specifici inviati da ciascun giurato per ogni sezione del form (header, participant, footer).
- **Generare una classifica** per una specifica giuria, selezionando i campi di tipo slider da includere nel calcolo del punteggio.
- **Esportare i risultati grezzi** in formato Excel.

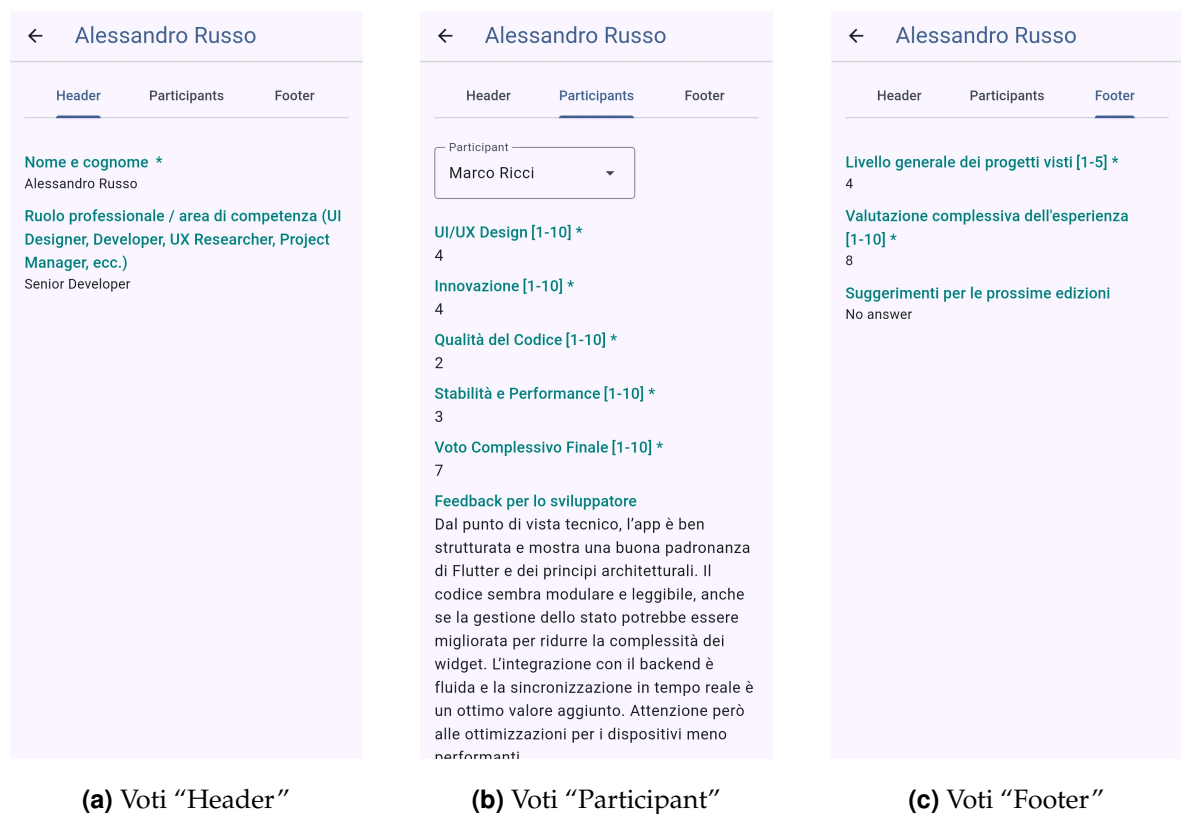


Figura 5.12: Visualizzazione dei voti inviati da un giurato, suddivisi per scope.

The screenshot shows a mobile application interface for a ranking system. At the top, there's a header with a back arrow and the title 'Ranking'. Below the header, there are two tabs: 'Edoardo De Angelis' and 'Elena Greco'. A dropdown menu labeled 'Select participant's form fields' is open, showing a list of criteria: 'UI/UX Design', 'Innovazione', 'Qualità del Codice', 'Stabilità e Performance', and 'Voto Complessivo Finale'. Below the dropdown, there's a section titled 'Generated Ranking' which displays a table of results. The table has three columns: a rank number in a blue circle, the participant's name and project, and the score. The participants are listed in descending order of score. To the right of the table, there are two buttons: 'Generate' (with a refresh icon) and 'Download' (with a download icon).

Rank	Participant	Score
1	Valentina Rizzo ArtQuest	696
2	Luca Bianchi RecipeFinder AI	688
3	Marco Ricci UniConnect	684
4	Laura Conti EventHive	
5	Davide Moretti CodeLeap	

Figura 5.13: La pagina per la generazione della classifica di una giuria.

5.6.4 Flusso del partecipante

Il percorso del partecipante è focalizzato sulla **sottomissione del proprio lavoro** prima della scadenza, tramite un form guidato a tre passaggi. Una volta completata, può visualizzare un riepilogo del lavoro caricato nella tab **Work**.

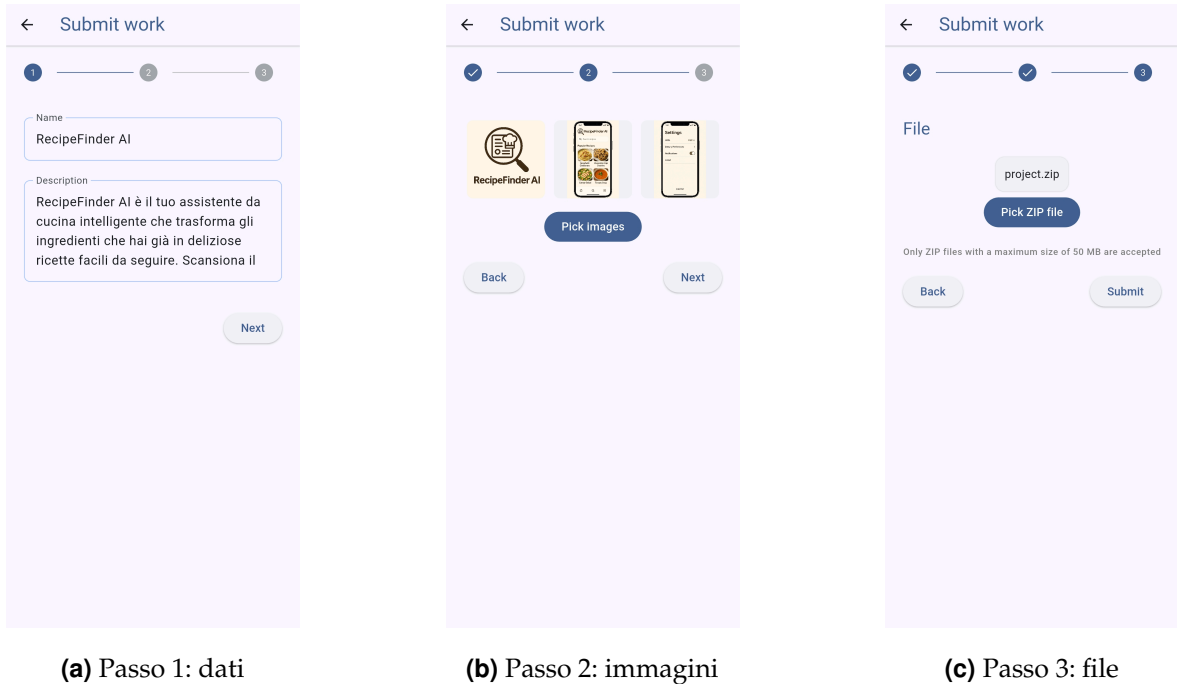


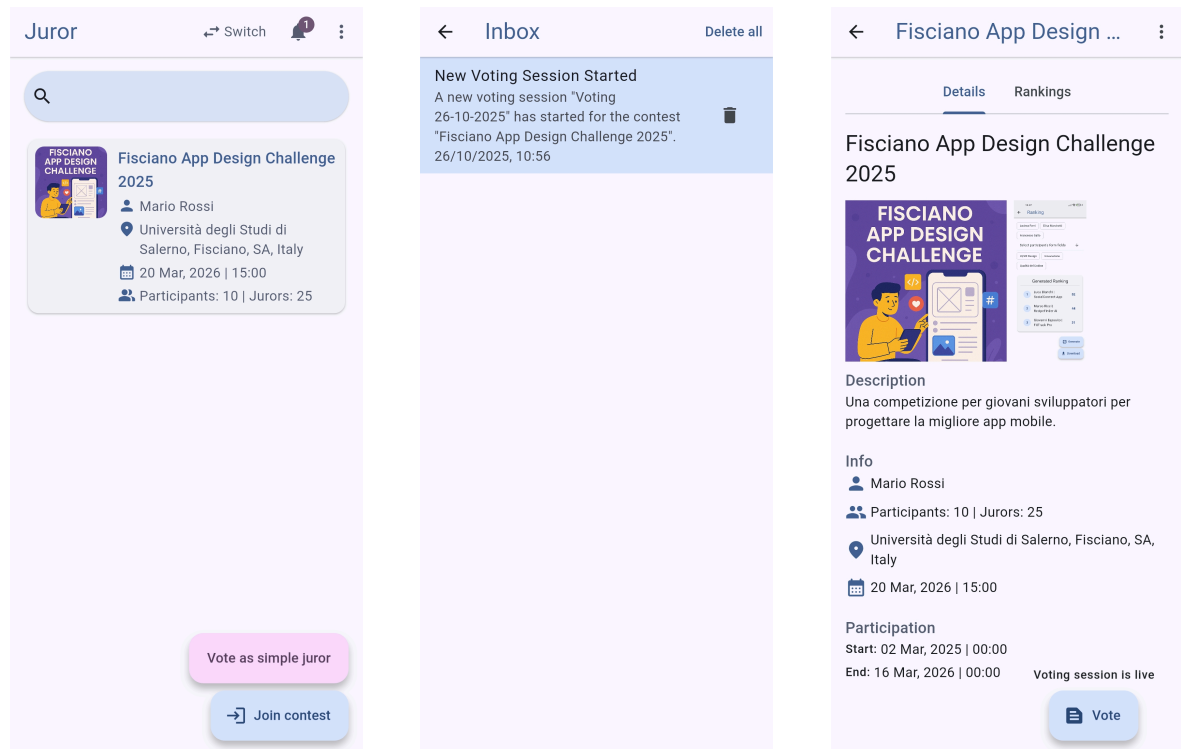
Figura 5.14: Le tre schermate del form guidato per la sottomissione di un lavoro.



Figura 5.15: La tab “Work” del partecipante dopo aver sottomesso il proprio lavoro.

5.6.5 Flusso del giurato

Un **giurato nominato**, dopo l'autenticazione, accede alla sua **Home Page**, dove visualizza i contest a cui è stato assegnato. Quando una sessione di voto è attiva, riceve una notifica nella sua **Inbox** e può accedere alla votazione dalla pagina di dettaglio del contest.



(a) Home page del giurato

(b) Notifica nella inbox

(c) Accesso alla votazione

Figura 5.16: Le schermate principali per il flusso del giurato nominato.

Procedura di votazione La procedura di votazione guida il giurato attraverso una serie di schermate strutturate: una **introduzione**, i campi **Header**, i campi **Participant** (uno per ogni opera) e i campi **Footer**.

← Voting

Valutazione Tecnica App

Benvenuto/a nella sessione di valutazione tecnica. Il tuo contributo è fondamentale per determinare la qualità, l'innovazione e la fattibilità dei progetti presentati. Ti chiediamo di valutare ogni progetto in modo obiettivo, basandoti sui criteri specificati in questo modulo. Il tuo feedback dettagliato aiuterà non solo a definire la classifica finale, ma fornirà anche spunti preziosi agli sviluppatori per migliorare il loro lavoro. Grazie per la tua partecipazione.

Verify Location

Next

← Voting

Header form

Nome e cognome *

Ruolo professionale / area di competenza (UI Designer, Developer, UX Researcher, Project Manager, ecc.)

Previous

Verify Location

Next

← Voting

RecipeFinder AI



Description
RecipeFinder AI è il tuo assistente da cucina intelligente che trasforma gli ingredienti che hai già in deliziose ricette facili da seguire. Scansiona il tuo frigo o la tua dispensa e lascia che la nostra IA trovi il pasto perfetto per te, riducendo gli sprechi alimentari e ispirando lo chef che è in te.

Luca Bianchi

Vote

UI/UX Design *

☐ ☐ ☐ ☐ ☐ ☒ ☐

1 2 3 4 5 6 7

Previous

Verify Location

Next

(a) Schermata introduttiva

(b) Votazione campi header

(c) Votazione campi partecipant

← Voting

Footer form

Livello generale dei progetti visti *

☐ ☐ ☐ ☒ ☐

1 2 3 4 5

Valutazione complessiva dell'esperienza *

☐ ☐ ☐ ☐ ☒ ☐ ☐

4 5 6 7 8 9 10

Suggerimenti per le prossime edizioni

Previous

Verify Location

Submit

(d) Votazione campi footer

Figura 5.17: Le quattro fasi principali della procedura di votazione guidata per il giurato.

5.6.6 Flusso del giurato semplice (ospite)

Il flusso “ospite” è progettato per essere il più rapido possibile. Dalla pagina di **Registrazione** o **Accesso**, un pulsante dedicato apre un popup per l’inserimento di **nome e cognome**. Una volta confermato, l’utente atterra su una **Home Page minimale**, da cui può accedere alla votazione tramite QR code o token.

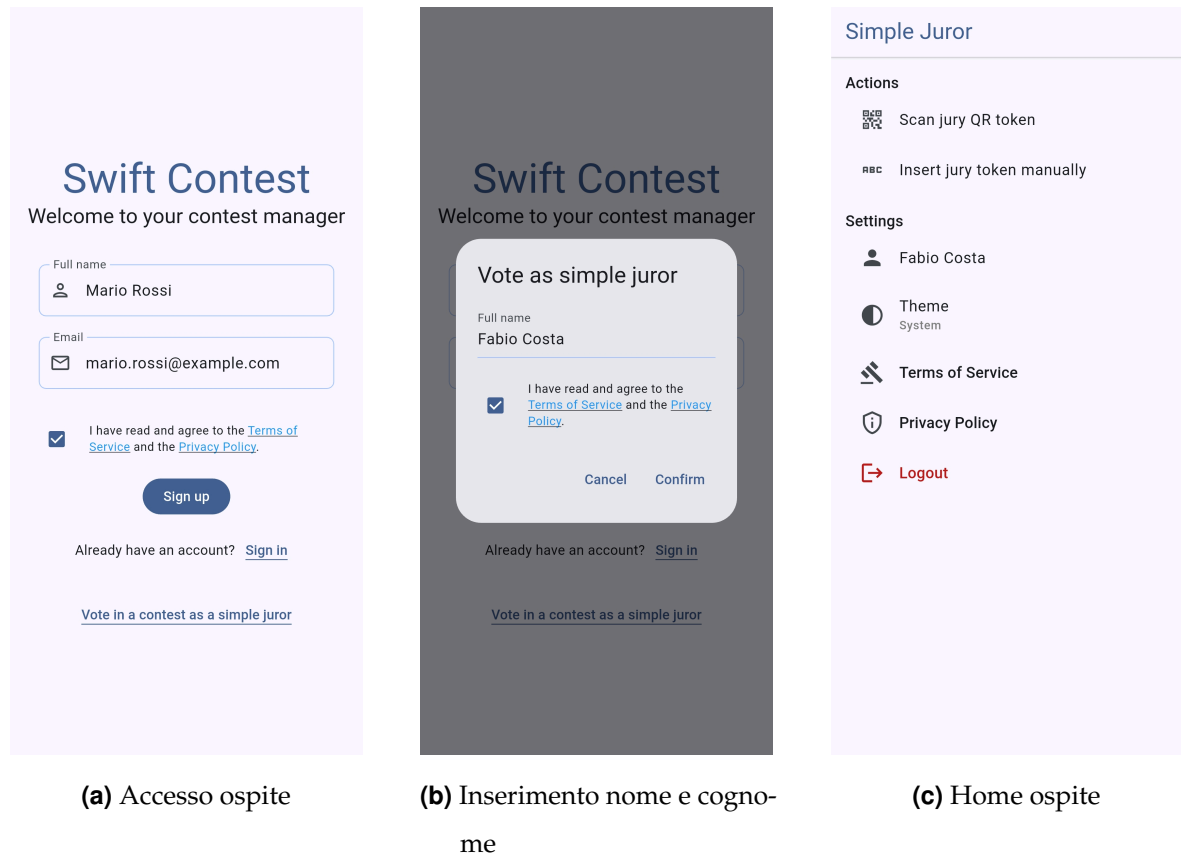


Figura 5.18: I tre passaggi per l’accesso di un giurato semplice (ospite) alla piattaforma.

Conclusioni

6.1 Sintesi del lavoro svolto

Il progetto di tesi ha portato alla progettazione, implementazione e distribuzione di **Swift Contest**, una piattaforma software completa e funzionante per la gestione di concorsi, che raggiunge con successo tutti gli obiettivi prefissati in fase di analisi. È stato dimostrato come, attraverso l'integrazione di un ecosistema tecnologico moderno e accessibile, sia possibile costruire un sistema complesso, sicuro e scalabile, capace di risolvere le inefficienze e le criticità dei processi di gestione tradizionali.

Il lavoro svolto, tuttavia, non si esaurisce con la presente tesi: esso costituisce una base di partenza per future iterazioni e, potenzialmente, per applicazioni in contesti reali, aprendo prospettive interessanti nel campo delle piattaforme collaborative e dei sistemi di votazione digitale.

6.2 Risultati ottenuti

I principali risultati ottenuti possono essere riassunti come segue:

- **Realizzazione di un'applicazione cross-platform unificata:** utilizzando Flutter, è stato sviluppato un unico client che offre un'esperienza utente coerente e

funzionale sia su piattaforme web che mobile (Android), validando l'efficacia di un approccio a codebase singola per ridurre i tempi di sviluppo e la complessità di manutenzione.

- **Implementazione di un'architettura backend sicura e robusta:** è stato costruito un backend basato sul Principio del Minimo Privilegio, dove l'accesso ai dati è interamente mediato da un'API controllata, proteggendo l'integrità del sistema.
- **Creazione di un sistema di votazione affidabile e trasparente:** il meccanismo di *snapshot* delle sessioni di voto garantisce l'immutabilità dei dati e l'auditabilità del processo, risolvendo una delle principali criticità dei sistemi di valutazione manuali.
- **Validazione di un'infrastruttura efficiente e sostenibile:** l'architettura basata su servizi cloud (Supabase, Vercel) e l'uso mirato di risorse onerose come il *realtime* dimostrano la fattibilità di costruire soluzioni complesse mantenendo i costi operativi contenuti, in linea con i piani gratuiti o a basso costo offerti dai provider.

Il prototipo realizzato non si limita a una dimostrazione tecnica, ma si configura come un'applicazione integrata e pronta per una fase di validazione sul campo, dimostrando la sua idoneità per contesti reali di piccola scala.

6.3 Conoscenze acquisite

Durante lo sviluppo del progetto Swift Contest, ho avuto l'opportunità di approfondire e consolidare una vasta gamma di conoscenze e competenze, sia a livello tecnico che metodologico. In particolare, ho acquisito una solida padronanza di:

- **Sviluppo Frontend con Flutter e Dart:** Ho imparato a progettare e implementare interfacce utente complesse e reattive, gestendo lo stato dell'applicazione tramite il pattern BLoC [18] e ottimizzando l'esperienza utente su diverse piattaforme. Ho approfondito l'uso di Dart Extensions per estendere le funzionalità di classi esistenti e la gestione della navigazione con AutoRoute [21].

- **Architetture Backend-as-a-Service (BaaS) con Supabase:** Ho compreso i vantaggi e le sfide di un'architettura BaaS, imparando a configurare e gestire un database PostgreSQL, a implementare logiche di business complesse tramite Funzioni RPC (PL/pgSQL) ed Edge Functions (TypeScript), e a sfruttare servizi come l'autenticazione, lo storage e il realtime.
- **Principi di Sicurezza del Software:** Ho applicato concretamente il Principio del Minimo Privilegio, progettando un accesso ai dati mediato da API e implementando meccanismi di autorizzazione robusti, inclusa la gestione dell'autenticazione anonima.
- **Design Pattern e Best Practice:** Ho utilizzato e compreso l'importanza di pattern come MVVM, BLoC, Singleton e la gestione centralizzata degli errori tramite il tipo Either, che contribuiscono a creare codice manutenibile e testabile.
- **Processi di Distribuzione e CI/CD:** Ho acquisito esperienza nella configurazione di pipeline di Continuous Integration/Continuous Deployment per applicazioni web e nella gestione sicura della distribuzione di APK Android tramite GitHub.
- **Modellazione del Dominio e Design del Database:** Ho imparato a tradurre requisiti complessi in un modello dati robusto, implementando strategie di integrità referenziale, cancellazione e snapshot dei dati per garantire consistenza e auditabilità.

Questa esperienza mi ha fornito una visione completa del ciclo di vita dello sviluppo software, dalla concezione all'implementazione e distribuzione.

6.4 Valutazione personale

La realizzazione di Swift Contest ha rappresentato un percorso di crescita significativo, non solo dal punto di vista tecnico, ma anche personale e progettuale. Affrontare la sfida di costruire un sistema completo da zero mi ha permesso di apprezzare l'importanza di un'**architettura ben ponderata** fin dalle prime fasi. In questo

contesto, ho deliberatamente scelto di non integrare funzionalità legate all'intelligenza artificiale, pur riconoscendone l'enorme importanza nel panorama tecnologico attuale. Per me era cruciale padroneggiare i **principi fondamentali** di un software "tradizionale" prima di aggiungere strati di complessità legati all'AI.

Se dovessi ripensare al progetto oggi, con l'esperienza maturata, dedicherei molta più attenzione alla **fase di progettazione iniziale**. All'inizio del percorso, non possedevo la base di conoscenze che ho oggi sulle strategie moderne per affrontare problemi complessi come la gestione dello stato, la sicurezza delle API o la distribuzione. Ho imparato sul campo che per progettare un sistema al meglio è fondamentale conoscere a fondo le alternative e i **pattern consolidati**.

Per questo motivo, sono estremamente soddisfatto del lavoro svolto, non solo per il risultato finale, ma soprattutto per il **valore formativo del processo**. Questo progetto mi ha insegnato a **valutare i trade-off** e a scegliere gli strumenti giusti per il problema giusto. Le competenze acquisite, in particolare nell'ambito dello sviluppo mobile e delle architetture backend, sono ora un bagaglio che posso trasferire con sicurezza nello sviluppo di qualsiasi software, sia che si tratti di utilizzare framework cross-platform alternativi come **Compose Multiplatform** [23] e **Kotlin Multiplatform** [24], sia che si tratti di approfondire lo sviluppo nativo, in particolare Android con **Jetpack Compose** [25].

Riferimenti

- [1] Google. "Flutter documentation," visitato il giorno 1 nov. 2025. indirizzo: <https://docs.flutter.dev/>
- [2] Supabase. "Supabase documentation," visitato il giorno 1 nov. 2025. indirizzo: <https://supabase.com/docs>
- [3] B. Adida. "Helios Voting," visitato il giorno 1 nov. 2025. indirizzo: <https://heliosvoting.org/>
- [4] Eventbrite, Inc. "Eventbrite," visitato il giorno 1 nov. 2025. indirizzo: <https://www.eventbrite.com/>
- [5] Google. "Firebase documentation," visitato il giorno 1 nov. 2025. indirizzo: <https://firebase.google.com/docs>
- [6] The PostgreSQL Global Development Group. "PostgreSQL documentation," visitato il giorno 1 nov. 2025. indirizzo: <https://www.postgresql.org/docs/>
- [7] Google. "Dart documentation," visitato il giorno 1 nov. 2025. indirizzo: <https://dart.dev/>
- [8] Microsoft. "TypeScript documentation," visitato il giorno 1 nov. 2025. indirizzo: <https://www.typescriptlang.org/docs/>

- [9] Vercel Inc. "Vercel," visitato il giorno 1 nov. 2025. indirizzo: <https://vercel.com/>
- [10] Resend, Inc. "Resend," visitato il giorno 1 nov. 2025. indirizzo: <https://resend.com/>
- [11] Google. "Google Places API documentation," visitato il giorno 1 nov. 2025. indirizzo: <https://developers.google.com/maps/documentation/places/web-service/overview>
- [12] JetBrains. "IntelliJ IDEA," visitato il giorno 1 nov. 2025. indirizzo: <https://www.jetbrains.com/idea/>
- [13] npm, Inc. "npm," visitato il giorno 1 nov. 2025. indirizzo: <https://www.npmjs.com/>
- [14] Retool, Inc. "Retool," visitato il giorno 1 nov. 2025. indirizzo: <https://retool.com/>
- [15] Software Freedom Conservancy. "Git," visitato il giorno 1 nov. 2025. indirizzo: <https://git-scm.com/>
- [16] GitHub, Inc. "GitHub," visitato il giorno 1 nov. 2025. indirizzo: <https://github.com/>
- [17] PlantUML. "PlantUML," visitato il giorno 1 nov. 2025. indirizzo: <https://plantuml.com/>
- [18] F. Angelov. "BLoC Library documentation," visitato il giorno 1 nov. 2025. indirizzo: <https://bloclibrary.dev/>
- [19] S. Maglione. "fpdart package documentation," visitato il giorno 1 nov. 2025. indirizzo: <https://pub.dev/packages/fpdart>
- [20] F. Angelov. "equatable package documentation," visitato il giorno 1 nov. 2025. indirizzo: <https://pub.dev/packages/equatable>
- [21] M. Akarie. "auto_route package documentation," visitato il giorno 1 nov. 2025. indirizzo: https://pub.dev/packages/auto_route
- [22] Google. "Material Design 3 documentation," visitato il giorno 1 nov. 2025. indirizzo: <https://m3.material.io/>

- [23] JetBrains. “Compose Multiplatform documentation,” visitato il giorno 1 nov. 2025. indirizzo: `https : / / www . jetbrains . com / lp / compose - multiplatform/`
- [24] JetBrains. “Kotlin Multiplatform documentation,” visitato il giorno 1 nov. 2025. indirizzo: `https://kotlinlang.org/docs/multiplatform.html`
- [25] Google. “Jetpack Compose documentation,” visitato il giorno 1 nov. 2025. indirizzo: `https://developer.android.com/jetpack/compose`

Ringraziamenti

Desidero esprimere la mia più sincera gratitudine a tutte le persone che, in modi diversi, hanno reso possibile la realizzazione di questo lavoro e hanno accompagnato il mio percorso.

Un ringraziamento speciale va alla mia relatrice, Professoressa Rita Francese, per avermi guidato con fiducia e discrezione. Le sono grato per avermi affidato un progetto che ha rappresentato non solo una sfida accademica, ma il primo, vero passo verso il percorso professionale che desidero intraprendere nello sviluppo di applicazioni mobile. Le sue intuizioni sono state fondamentali per dare una direzione a questo lavoro.

Il ringraziamento più profondo e sentito va alla mia famiglia, che non è semplicemente un supporto, ma il centro del mio universo e il mio rifugio più prezioso.

A mia madre e a mio padre, che siete e sempre sarete il mio porto sicuro. Grazie per avermi insegnato il valore del lavoro e della perseveranza, ma soprattutto per averlo fatto con un amore incondizionato che non ha mai chiesto nulla in cambio. I vostri sacrifici silenziosi, i gesti quotidiani e il vostro instancabile supporto mi hanno garantito la serenità necessaria per dedicarmi completamente a questo percorso, libero dalle pressioni delle aspettative e sostenuto solo dalla leggerezza del vostro incoraggiamento.

Alle mie sorelle, Caterina e Carmela. Entrambe più grandi, avete segnato il mio percorso in modi unici e insostituibili. Carmela, con la tua vicinanza d'età, sei stata per me come una gemella, una complice naturale in ogni avventura e risata. Caterina, con la tua maggiore esperienza, sei stata una guida preziosa nei momenti di incertezza. Insieme, siete state l'antidoto più efficace alla solitudine che a volte accompagna questo percorso, riempiendo le mie giornate con il suono delle risate, e persino con quelle litigate che, a modo loro, sono sempre state la prova di un legame che non ha bisogno di filtri per essere autentico.

In questa famiglia, un posto d'onore spetta alle presenze insostituibili che non usano parole, ma comunicano un affetto puro: Stella, Tino e, soprattutto, Fiocco.

Al mio gatto Fiocco, senza esagerazione, la mia anima gemella. Sordo ai rumori del mondo, ma mai silenzioso, mi hai insegnato che la comunicazione più autentica trova sempre la sua voce. Il tuo esserci, presente e tranquillo, è stato il conforto più grande nei momenti di maggiore stress e fatica.

A tutti voi, che avete reso possibile questo viaggio, dedico questo traguardo.